RGA IM2D API Instruction

ID: RK-KF-YF-403

Release Version: V2.2.7

Release Date: 2025-04-30

Security Level: □Top-Secret □Secret □Internal ■Public

Rockchip Electronics Co., Ltd.

No.18 Building, A District, No.89, software Boulevard Fuzhou, Fujian,PRC

Website: www.rock-chips.com

Customer service Tel: +86-4007-700-590

Customer service Fax: +86-591-83951833

Customer service e-Mail: fae@rock-chips.com

Intended Audience

This document (this guide) is mainly intended for:

- Technical support engineers
- Software development engineers

Revision History

Date	Version	Author	Revision Description
2020/06/24	1.0.0	Chen Cheng, Li Huang	Initial version.
2020/10/16	1.0.1	Chen Cheng, Li Huang, Yu Qiaowei	Update part of the APIs.
2021/12/07	2.0.0	Chen Cheng, Li Huang, Yu Qiaowei	Add RGA3 related support.
2022/01/20	2.1.0	Chen Cheng, Li Huang, Yu Qiaowei	- Update im2d api description. - Updated hardware index description and alignment restrictions. - Add data structure description.
2022/01/20	2.1.1	Chen Cheng, Li Huang, Yu Qiaowei	Supplemental formatting support/alignment instructions.
2022/09/15	2.2.0	Chen Cheng, Li Huang, Yu Qiaowei	- Supplementary default value description - New array api - New task api - New rectangle border drawing api
2023/02/09	2.2.1	Yu Qiaowei	Format document.
2023/06/28	2.2.2	Yu Qiaowei	- Add chip RK3562 introduction - Improve the precautions for grayscale images
2024/03/06	2.2.3	Yu Qiaowei	Add chip RK3576 introduction
2024/08/22	2.2.4	Yu Qiaowei	Add chip RK3506、RV1103B introduction
2024/11/18	2.2.5	Yu Qiaowei	Add the support status of APIs in different environments.
2025/03/27	2.2.6	Yu Qiaowei	Add chip RV1126B introduction
2025/04/30	2.2.7	Yu Qiaowei	Fixed over-constraints on actual_height in the description of RK3506/RV1103B

Contents

1. Introductions

RGA (Raster Graphic Acceleration Unit) is an independent 2D hardware accelerator that can be used to speed up point/line drawing, perform image resizing, rotation, bitBlt, alpha blending and other common 2D graphics operations.

1.1 Design Index

Version	Codename	Chip	Source		Destination		Function	Pixels/C
			min	max	min	max		
RGA1	Pagani	RK3066	2x2	8192x8192	2x2	2048x2048	90/180/270 Rotate X/Y Mirror Crop 1/2~8 scale Alpha blend Color key Color fill ROP IOMMU(32bit)	1
	Jaguar Plus	RK3188						
	Beetles	RK2926/2928						
	Beetles Plus	RK3026/3028						
RGA1_plus	Audi	RK3128	2x2	8192x8192	2x2	2048x2048	90/180/270 Rotate X/Y Mirror Crop 1/2~8 scale Alpha blend Color key Color fill Color palette IOMMU(32bit)	1
	Granite	Sofia 3gr						
RGA2	Lincoln	RK3288/3288w	2x2	8192x8192	2x2	4096x4096	90/180/270 Rotate X/Y Mirror Crop 1/16~16 scale scale-up(bi-linear/bi-cubic) scale-down(average) Alpha blend Color key Color fill Color palette ROP IOMMU(32bit)	2
	Capricorn	RK3190						
RGA2-Lite0	Maybach	RK3368	2x2	8192x8192	2x2	4096x4096	90/180/270 Rotate X/Y Mirror Crop 1/8~8 scale scale-up(bi-linear/bi-cubic) scale-down(average) Alpha blend Color key Color fill Color palette ROP IOMMU(32bit)	2
	BMW	RK3366						

RGA2-Lite1	Benz	RK3228	2x2	8192x8192	2x2	4096x4096	90/180/270 Rotate X/Y Mirror Crop 1/8~8 scale scale-up(bi-linear/bi-cubic) scale-down(average) Alpha blend Color key Color fill Color palette IOMMU(32bit)	2
	Infiniti	RK3228H						
	Gemini	RK3326						
	Lion	RK1808						
RGA2-Lite2	Libra	RK3506	2x2	1280x8192	2x2	1280x4096	90/180/270 Rotate X/Y Mirror Crop 1/16~16 scale scale-up(bi-linear/bi-cubic) scale-down(average) Alpha blend Color key Color fill Color palette Guassion blur alpha-8bit	2
RGA2-Lite3	Pather	RK1103B	2x2	2880x8192	2x2	2880x8192	90/180/270 Rotate X/Y Mirror Crop 1/16~16 scale scale-up(bi-linear) scale-down(bi-linear/average) Color fill	2
RGA2-Enhance	Mclaren	RK3399	2x2	8192x8192	2x2	4096x4096	90/180/270 Rotate X/Y Mirror Crop 1/16~16 scale scale-up(bi-linear/bi-cubic) scale-down(average) Alpha blend Color key Color fill Color palette ROP(NA for RV1108/RV1109/RK3566) NN quantize(NA for RK3399/RV1108) osd (only RV1106/RV1103/RK3562/RK3528) mosaic(only RV1106/RV1103/RK3562/RK3528) IOMMU(32bit, RK3528/RK3562为40bit，NA for RV1106/1103)	2
	Mercury	RK1108						
	Puma	RV1126/RV1109						
	skylarkV2	RK3566/RK3568						
	Orion	RK3588						
	Otter	RV1106/1103						
	Bull	RK3528						
	Snipe	RK3562						

RGA2-Pro	Heron	RK3576	2x2	8192x8192	2x2	8192x8192	90/180/270 Rotate X/Y Mirror Crop 1/16~16 scale scale-up(bi-linear/bi-cubic) scale-down(bi-linear/average) Alpha blend Color key Color fill Color palette ROP osd mosaic(only RK3576) ARGB5551 alpha bit map rkfbc64x4 input(only RK3576) afbc32x8 input(split mode, only RK3576) tile4x4(RV1126B only input) IOMMU(40bit)	2
	Swan	RV1126B						
RGA3	Orion	RK3588	68x2	8176x8176	68x2	8128x8128	90/180/270 Rotate X/Y Mirror Crop 1/8~8 scale scale-up(bi-cubic) scale-down(average) Alpha blend Color key afbc16x16 IOMMU(40bit)	3 (by pa 2 (scale)

> Note:
>
> 1). The capabilities of Pixels/cycle are theoretical data, and the actual operating performance is related to bandwidth, hardware frequency, etc. The list data is for reference only.
>
> 2). In addition to the minimum input resolution limit, the x, y, width, and height parameters of the actual operation rectangle that can be set for each channel must be greater than or equal to 2.
>
> 3). The addressing capability of RGA is related to the number of bits of IOMMU. For example, the actual physical addressing capability of RGA equipped with 32bit IOMMU only supports 0~4G memory space.

1.2 Image Format Supported

- Pixel Format conversion, BT.601/BT.709/BT.2020(only RGA3)
- Dither operation

Version	Codename	Chip	Input Data Format	Output Data Format
RGA1	Pagani	RK3066	RK_FORMAT_RGBA_8888 RK_FORMAT_BGRA_8888 RK_FORMAT_ARGB_8888 RK_FORMAT_ABGR_8888 RK_FORMAT_RGBX_8888 RK_FORMAT_BGRX_8888 RK_FORMAT_XRGB_8888 RK_FORMAT_XBGR_8888 RK_FORMAT_ARGB_4444 RK_FORMAT_ABGR_4444 RK_FORMAT_ARGB_5551 RK_FORMAT_ABGR_5551 RK_FORMAT_RGB_888 RK_FORMAT_BGR_888 RK_FORMAT_RGB_565 RK_FORMAT_BGR_565 RK_FORMAT_YCbCr_420_SP RK_FORMAT_YCrCb_420_SP RK_FORMAT_YCbCr_422_SP RK_FORMAT_YCrCb_422_SP RK_FORMAT_YCbCr_420_P RK_FORMAT_YCrCb_420_P RK_FORMAT_YCbCr_422_P RK_FORMAT_YCrCb_422_P RK_FORMAT_BPP1 RK_FORMAT_BPP2 RK_FORMAT_BPP4 RK_FORMAT_BPP8	RK_FORMAT_RGBA_8888 RK_FORMAT_BGRA_8888 RK_FORMAT_ARGB_8888 RK_FORMAT_ABGR_8888 RK_FORMAT_RGBX_8888 RK_FORMAT_BGRX_8888 RK_FORMAT_XRGB_8888 RK_FORMAT_XBGR_8888 RK_FORMAT_ARGB_4444 RK_FORMAT_ABGR_4444 RK_FORMAT_ARGB_5551 RK_FORMAT_ABGR_5551 RK_FORMAT_RGB_888 RK_FORMAT_BGR_888 RK_FORMAT_RGB_565 RK_FORMAT_BGR_565 RK_FORMAT_YCbCr_420_SP (only for Blur/sharpness) RK_FORMAT_YCrCb_420_SP (only for Blur/sharpness) RK_FORMAT_YCbCr_422_SP (only for Blur/sharpness) RK_FORMAT_YCrCb_422_SP (only for Blur/sharpness) RK_FORMAT_YCbCr_420_P (only for Blur/sharpness) RK_FORMAT_YCrCb_420_P (only for Blur/sharpness) RK_FORMAT_YCbCr_422_P (only for Blur/sharpness) RK_FORMAT_YCrCb_422_P (only for Blur/sharpness)
	Jaguar Plus	RK3188		
	Beetles	RK2926/2928		
	Beetles Plus	RK3026/3028		

RGA1_plus	Audi	RK3128	RK_FORMAT_RGBA_8888 RK_FORMAT_BGRA_8888 RK_FORMAT_ARGB_8888 RK_FORMAT_ABGR_8888 RK_FORMAT_RGBX_8888 RK_FORMAT_BGRX_8888 RK_FORMAT_XRGB_8888 RK_FORMAT_XBGR_8888 RK_FORMAT_ARGB_4444 RK_FORMAT_ABGR_4444 RK_FORMAT_ARGB_5551 RK_FORMAT_ABGR_5551 RK_FORMAT_RGB_888 RK_FORMAT_BGR_888 RK_FORMAT_RGB_565 RK_FORMAT_BGR_565 RK_FORMAT_YCbCr_420_SP RK_FORMAT_YCrCb_420_SP RK_FORMAT_YCbCr_422_SP RK_FORMAT_YCrCb_422_SP RK_FORMAT_YCbCr_420_P RK_FORMAT_YCrCb_420_P RK_FORMAT_YCbCr_422_P RK_FORMAT_YCrCb_422_P RK_FORMAT_BPP1 RK_FORMAT_BPP2 RK_FORMAT_BPP4 RK_FORMAT_BPP8	RK_FORMAT_RGBA_8888 RK_FORMAT_BGRA_8888 RK_FORMAT_ARGB_8888 RK_FORMAT_ABGR_8888 RK_FORMAT_RGBX_8888 RK_FORMAT_BGRX_8888 RK_FORMAT_XRGB_8888 RK_FORMAT_XBGR_8888 RK_FORMAT_ARGB_4444 RK_FORMAT_ABGR_4444 RK_FORMAT_ARGB_5551 RK_FORMAT_ABGR_5551 RK_FORMAT_RGB_888 RK_FORMAT_BGR_888 RK_FORMAT_RGB_565 RK_FORMAT_BGR_565 RK_FORMAT_YCbCr_420_SP (only for normal Bitblt without alpha) RK_FORMAT_YCrCb_420_SP (only for normal Bitblt without alpha) RK_FORMAT_YCbCr_422_SP (only for normal Bitblt without alpha) RK_FORMAT_YCrCb_422_SP (only for normal Bitblt without alpha) RK_FORMAT_YCbCr_420_P (only for normal Bitblt without alpha) RK_FORMAT_YCrCb_420_P (only for normal Bitblt without alpha) RK_FORMAT_YCbCr_422_P (only for normal Bitblt without alpha) RK_FORMAT_YCrCb_422_P (only for normal Bitblt without alpha)
	Granite	Sofia 3gr		

RGA2	Lincoln	RK3288/3288w	RK_FORMAT_RGBA_8888 RK_FORMAT_BGRA_8888 RK_FORMAT_ARGB_8888 RK_FORMAT_ABGR_8888 RK_FORMAT_RGBX_8888 RK_FORMAT_BGRX_8888 RK_FORMAT_XRGB_8888 RK_FORMAT_XBGR_8888 RK_FORMAT_ARGB_4444 RK_FORMAT_ABGR_4444 RK_FORMAT_ARGB_5551 RK_FORMAT_ABGR_5551 RK_FORMAT_RGB_888 RK_FORMAT_BGR_888 RK_FORMAT_RGB_565 RK_FORMAT_BGR_565 RK_FORMAT_YCbCr_420_SP RK_FORMAT_YCrCb_420_SP RK_FORMAT_YCbCr_422_SP RK_FORMAT_YCrCb_422_SP RK_FORMAT_YCbCr_420_P RK_FORMAT_YCrCb_420_P RK_FORMAT_YCbCr_422_P RK_FORMAT_YCrCb_422_P RK_FORMAT_BPP1 (only for color palette) RK_FORMAT_BPP2 (only for color palette) RK_FORMAT_BPP4 (only for color palette) RK_FORMAT_BPP8 (only for color palette)	RK_FORMAT_RGBA_8888 RK_FORMAT_BGRA_8888 RK_FORMAT_ARGB_8888 RK_FORMAT_ABGR_8888 RK_FORMAT_RGBX_8888 RK_FORMAT_BGRX_8888 RK_FORMAT_XRGB_8888 RK_FORMAT_XBGR_8888 RK_FORMAT_ARGB_4444 RK_FORMAT_ABGR_4444 RK_FORMAT_ARGB_5551 RK_FORMAT_ABGR_5551 RK_FORMAT_RGB_888 RK_FORMAT_BGR_888 RK_FORMAT_RGB_565 RK_FORMAT_BGR_565 RK_FORMAT_YCbCr_420_SP RK_FORMAT_YCrCb_420_SP RK_FORMAT_YCbCr_422_SP RK_FORMAT_YCrCb_422_SP RK_FORMAT_YCbCr_420_P RK_FORMAT_YCrCb_420_P RK_FORMAT_YCbCr_422_P RK_FORMAT_YCrCb_422_P
	Capricorn	RK3190		

RGA2-Lite0	Maybach	RK3368	RK_FORMAT_RGBA_8888 RK_FORMAT_BGRA_8888 RK_FORMAT_ARGB_8888 RK_FORMAT_ABGR_8888 RK_FORMAT_RGBX_8888 RK_FORMAT_BGRX_8888 RK_FORMAT_XRGB_8888 RK_FORMAT_XBGR_8888 RK_FORMAT_ARGB_4444 RK_FORMAT_ABGR_4444 RK_FORMAT_ARGB_5551 RK_FORMAT_ABGR_5551 RK_FORMAT_RGB_888 RK_FORMAT_BGR_888 RK_FORMAT_RGB_565 RK_FORMAT_BGR_565 RK_FORMAT_YCbCr_420_SP RK_FORMAT_YCrCb_420_SP RK_FORMAT_YCbCr_422_SP RK_FORMAT_YCrCb_422_SP RK_FORMAT_YCbCr_420_P RK_FORMAT_YCrCb_420_P RK_FORMAT_YCbCr_422_P RK_FORMAT_YCrCb_422_P RK_FORMAT_BPP1 (only for color palette) RK_FORMAT_BPP2 (only for color palette) RK_FORMAT_BPP4 (only for color palette) RK_FORMAT_BPP8 (only for color palette)	RK_FORMAT_RGBA_8888 RK_FORMAT_BGRA_8888 RK_FORMAT_ARGB_8888 RK_FORMAT_ABGR_8888 RK_FORMAT_RGBX_8888 RK_FORMAT_BGRX_8888 RK_FORMAT_XRGB_8888 RK_FORMAT_XBGR_8888 RK_FORMAT_ARGB_4444 RK_FORMAT_ABGR_4444 RK_FORMAT_ARGB_5551 RK_FORMAT_ABGR_5551 RK_FORMAT_RGB_888 RK_FORMAT_BGR_888 RK_FORMAT_RGB_565 RK_FORMAT_BGR_565 RK_FORMAT_YCbCr_420_SP RK_FORMAT_YCrCb_420_SP RK_FORMAT_YCbCr_422_SP RK_FORMAT_YCrCb_422_SP RK_FORMAT_YCbCr_420_P RK_FORMAT_YCrCb_420_P RK_FORMAT_YCbCr_422_P RK_FORMAT_YCrCb_422_P
	BMW	RK3366		

RGA2-Lite1	Benz	RK3228	RK_FORMAT_RGBA_8888 RK_FORMAT_BGRA_8888 RK_FORMAT_ARGB_8888 RK_FORMAT_ABGR_8888 RK_FORMAT_RGBX_8888 RK_FORMAT_BGRX_8888 RK_FORMAT_XRGB_8888 RK_FORMAT_XBGR_8888 RK_FORMAT_RGBA_4444 RK_FORMAT_ARGB_5551 RK_FORMAT_ABGR_5551 RK_FORMAT_RGB_888 RK_FORMAT_BGR_888 RK_FORMAT_RGB_565 RK_FORMAT_BGR_565 RK_FORMAT_YCbCr_420_SP RK_FORMAT_YCrCb_420_SP RK_FORMAT_YCbCr_422_SP RK_FORMAT_YCrCb_422_SP RK_FORMAT_YCbCr_420_P RK_FORMAT_YCrCb_420_P RK_FORMAT_YCbCr_422_P RK_FORMAT_YCrCb_422_P RK_FORMAT_YCbCr_420_SP_10B RK_FORMAT_YCrCb_420_SP_10B RK_FORMAT_YCbCr_422_SP_10B RK_FORMAT_YCrCb_422_SP_10B RK_FORMAT_BPP1 (only for color palette) RK_FORMAT_BPP2 (only for color palette) RK_FORMAT_BPP4 (only for color palette) RK_FORMAT_BPP8 (only for color palette)	RK_FORMAT_RGBA_8888 RK_FORMAT_BGRA_8888 RK_FORMAT_ARGB_8888 RK_FORMAT_ABGR_8888 RK_FORMAT_RGBX_8888 RK_FORMAT_BGRX_8888 RK_FORMAT_XRGB_8888 RK_FORMAT_XBGR_8888 RK_FORMAT_ARGB_4444 RK_FORMAT_ABGR_4444 RK_FORMAT_ARGB_5551 RK_FORMAT_ABGR_5551 RK_FORMAT_RGB_888 RK_FORMAT_BGR_888 RK_FORMAT_RGB_565 RK_FORMAT_BGR_565 RK_FORMAT_YCbCr_420_SP RK_FORMAT_YCrCb_420_SP RK_FORMAT_YCbCr_422_SP RK_FORMAT_YCrCb_422_SP RK_FORMAT_YCbCr_420_P RK_FORMAT_YCrCb_420_P RK_FORMAT_YCbCr_422_P RK_FORMAT_YCrCb_422_P
	Infiniti	RK3228H		
	Gemini	RK3326		
	Lion	RK1808		

RGA2-Lite2	Libra	RK3506	RK_FORMAT_RGBA_8888 RK_FORMAT_BGRA_8888 RK_FORMAT_ARGB_8888 RK_FORMAT_ABGR_8888 RK_FORMAT_RGBX_8888 RK_FORMAT_BGRX_8888 RK_FORMAT_XRGB_8888 RK_FORMAT_XBGR_8888 RK_FORMAT_ARGB_4444 RK_FORMAT_ABGR_4444 RK_FORMAT_ARGB_5551 RK_FORMAT_ABGR_5551 RK_FORMAT_RGB_888 RK_FORMAT_BGR_888 RK_FORMAT_RGB_565 RK_FORMAT_BGR_565 RK_FORMAT_YCbCr_420_SP RK_FORMAT_YCrCb_420_SP RK_FORMAT_YCbCr_422_SP RK_FORMAT_YCrCb_422_SP RK_FORMAT_YCbCr_420_P RK_FORMAT_YCrCb_420_P RK_FORMAT_YCbCr_422_P RK_FORMAT_YCrCb_422_P RK_FORMAT_YUYV_422 RK_FORMAT_YVYU_422 RK_FORMAT_UYVY_422 RK_FORMAT_VYUY_422 RK_FORMAT_YCbCr_400 RK_FORMAT_YCbCr_420_SP_10B RK_FORMAT_YCrCb_420_SP_10B RK_FORMAT_YCbCr_422_SP_10B RK_FORMAT_YCrCb_422_SP_10B RK_FORMAT_A8 RK_FORMAT_BPP1 (only for color palette) RK_FORMAT_BPP2 (only for color palette) RK_FORMAT_BPP4 (only for color palette) RK_FORMAT_BPP8 (only for color palette)	RK_FORMAT_RGBA_8888 RK_FORMAT_BGRA_8888 RK_FORMAT_ARGB_8888 RK_FORMAT_ABGR_8888 RK_FORMAT_RGBX_8888 RK_FORMAT_BGRX_8888 RK_FORMAT_XRGB_8888 RK_FORMAT_XBGR_8888 RK_FORMAT_ARGB_4444 RK_FORMAT_ABGR_4444 RK_FORMAT_ARGB_5551 RK_FORMAT_ABGR_5551 RK_FORMAT_RGB_888 RK_FORMAT_BGR_888 RK_FORMAT_RGB_565 RK_FORMAT_BGR_565 RK_FORMAT_YCbCr_420_SP RK_FORMAT_YCrCb_420_SP RK_FORMAT_YCbCr_422_SP RK_FORMAT_YCrCb_422_SP RK_FORMAT_YCbCr_420_P RK_FORMAT_YCrCb_420_P RK_FORMAT_YCbCr_422_P RK_FORMAT_YCrCb_422_P RK_FORMAT_YUYV_422 RK_FORMAT_YVYU_422 RK_FORMAT_UYVY_422 RK_FORMAT_VYUY_422 RK_FORMAT_YCbCr_400 RK_FORMAT_Y4

RGA2-Lite2	Pather	RV1103B	RK_FORMAT_RGBX_8888 RK_FORMAT_BGRX_8888 RK_FORMAT_XRGB_8888 RK_FORMAT_XBGR_8888 RK_FORMAT_RGB_888 RK_FORMAT_BGR_888 RK_FORMAT_RGB_565 RK_FORMAT_BGR_565 RK_FORMAT_YCbCr_420_SP RK_FORMAT_YCrCb_420_SP RK_FORMAT_YCbCr_422_SP RK_FORMAT_YCrCb_422_SP RK_FORMAT_YCbCr_420_P RK_FORMAT_YCrCb_420_P RK_FORMAT_YCbCr_422_P RK_FORMAT_YCrCb_422_P RK_FORMAT_YUYV_422 RK_FORMAT_YVYU_422 RK_FORMAT_UYVY_422 RK_FORMAT_VYUY_422 RK_FORMAT_YCbCr_400	RK_FORMAT_RGBX_8888 RK_FORMAT_BGRX_8888 RK_FORMAT_XRGB_8888 RK_FORMAT_XBGR_8888 RK_FORMAT_RGB_888 RK_FORMAT_BGR_888 RK_FORMAT_RGB_565 RK_FORMAT_BGR_565 RK_FORMAT_YCbCr_420_SP RK_FORMAT_YCrCb_420_SP RK_FORMAT_YCbCr_422_SP RK_FORMAT_YCrCb_422_SP RK_FORMAT_YCbCr_420_P RK_FORMAT_YCrCb_420_P RK_FORMAT_YCbCr_422_P RK_FORMAT_YCrCb_422_P RK_FORMAT_YUYV_422 RK_FORMAT_YVYU_422 RK_FORMAT_UYVY_422 RK_FORMAT_VYUY_422 RK_FORMAT_YCbCr_400

RGA2-Enhance	Mclaren	RK3399	RK_FORMAT_RGBA_8888 RK_FORMAT_BGRA_8888 RK_FORMAT_ARGB_8888 RK_FORMAT_ABGR_8888 RK_FORMAT_RGBX_8888 RK_FORMAT_BGRX_8888 RK_FORMAT_XRGB_8888 RK_FORMAT_XBGR_8888 RK_FORMAT_ARGB_4444 RK_FORMAT_ABGR_4444 RK_FORMAT_ARGB_5551 RK_FORMAT_ABGR_5551 RK_FORMAT_RGB_888 RK_FORMAT_BGR_888 RK_FORMAT_RGB_565 RK_FORMAT_BGR_565 RK_FORMAT_YCbCr_420_SP RK_FORMAT_YCrCb_420_SP RK_FORMAT_YCbCr_422_SP RK_FORMAT_YCrCb_422_SP RK_FORMAT_YCbCr_420_P RK_FORMAT_YCrCb_420_P RK_FORMAT_YCbCr_422_P RK_FORMAT_YCrCb_422_P RK_FORMAT_YCbCr_420_SP_10B RK_FORMAT_YCrCb_420_SP_10B RK_FORMAT_YCbCr_422_SP_10B RK_FORMAT_YCrCb_422_SP_10B RK_FORMAT_BPP1 (only for color palette) RK_FORMAT_BPP2 (only for color palette) RK_FORMAT_BPP4 (only for color palette) RK_FORMAT_BPP8 (only for color palette)	RK_FORMAT_RGBA_8888 RK_FORMAT_BGRA_8888 RK_FORMAT_ARGB_8888 RK_FORMAT_ABGR_8888 RK_FORMAT_RGBX_8888 RK_FORMAT_BGRX_8888 RK_FORMAT_XRGB_8888 RK_FORMAT_XBGR_8888 RK_FORMAT_ARGB_4444 RK_FORMAT_ABGR_4444 RK_FORMAT_ARGB_5551 RK_FORMAT_ABGR_5551 RK_FORMAT_RGB_888 RK_FORMAT_BGR_888 RK_FORMAT_RGB_565 RK_FORMAT_BGR_565 RK_FORMAT_YCbCr_420_SP RK_FORMAT_YCrCb_420_SP RK_FORMAT_YCbCr_422_SP RK_FORMAT_YCrCb_422_SP RK_FORMAT_YCbCr_420_P RK_FORMAT_YCrCb_420_P RK_FORMAT_YCbCr_422_P RK_FORMAT_YCrCb_422_P RK_FORMAT_YUYV_422 RK_FORMAT_YVYU_422 RK_FORMAT_UYVY_422 RK_FORMAT_VYUY_422
	Mercury	RK1108		

	Puma	RV1126/ RV1109	RK_FORMAT_RGBA_8888 RK_FORMAT_BGRA_8888 RK_FORMAT_ARGB_8888 RK_FORMAT_ABGR_8888 RK_FORMAT_RGBX_8888 RK_FORMAT_BGRX_8888 RK_FORMAT_XRGB_8888 RK_FORMAT_XBGR_8888 RK_FORMAT_ARGB_4444 RK_FORMAT_ABGR_4444 RK_FORMAT_ARGB_5551 RK_FORMAT_ABGR_5551 RK_FORMAT_RGB_888 RK_FORMAT_BGR_888 RK_FORMAT_RGB_565 RK_FORMAT_BGR_565 RK_FORMAT_YCbCr_420_SP RK_FORMAT_YCrCb_420_SP RK_FORMAT_YCbCr_422_SP RK_FORMAT_YCrCb_422_SP RK_FORMAT_YCbCr_420_P RK_FORMAT_YCrCb_420_P RK_FORMAT_YCbCr_422_P RK_FORMAT_YCrCb_422_P RK_FORMAT_YUYV_422 RK_FORMAT_YVYU_422 RK_FORMAT_UYVY_422 RK_FORMAT_VYUY_422 RK_FORMAT_YCbCr_400 RK_FORMAT_YCbCr_420_SP_10B RK_FORMAT_YCrCb_420_SP_10B RK_FORMAT_YCbCr_422_SP_10B RK_FORMAT_YCrCb_422_SP_10B RK_FORMAT_BPP1 (only for color palette) RK_FORMAT_BPP2 (only for color palette) RK_FORMAT_BPP4 (only for color palette) RK_FORMAT_BPP8 (only for color palette)	RK_FORMAT_RGBA_8888 RK_FORMAT_BGRA_8888 RK_FORMAT_ARGB_8888 RK_FORMAT_ABGR_8888 RK_FORMAT_RGBX_8888 RK_FORMAT_BGRX_8888 RK_FORMAT_XRGB_8888 RK_FORMAT_XBGR_8888 RK_FORMAT_ARGB_4444 RK_FORMAT_ABGR_4444 RK_FORMAT_ARGB_5551 RK_FORMAT_ABGR_5551 RK_FORMAT_RGB_888 RK_FORMAT_BGR_888 RK_FORMAT_RGB_565 RK_FORMAT_BGR_565 RK_FORMAT_YCbCr_420_SP RK_FORMAT_YCrCb_420_SP RK_FORMAT_YCbCr_422_SP RK_FORMAT_YCrCb_422_SP RK_FORMAT_YCbCr_420_P RK_FORMAT_YCrCb_420_P RK_FORMAT_YCbCr_422_P RK_FORMAT_YCrCb_422_P RK_FORMAT_YUYV_422 RK_FORMAT_YVYU_422 RK_FORMAT_UYVY_422 RK_FORMAT_VYUY_422 RK_FORMAT_YCbCr_400 RK_FORMAT_Y4
	skylarkV2	RK3566/RK3568		
	Orion	RK3588		
	Otter	RV1106/1103		
	Bull	RK3528		
	Snipe	RK3562		

RGA2-Pro	Heron	RK3576	RK_FORMAT_RGBA_8888 RK_FORMAT_BGRA_8888 RK_FORMAT_ARGB_8888 RK_FORMAT_ABGR_8888 RK_FORMAT_RGBX_8888 RK_FORMAT_BGRX_8888 RK_FORMAT_XRGB_8888 RK_FORMAT_XBGR_8888 RK_FORMAT_ARGB_4444 RK_FORMAT_ABGR_4444 RK_FORMAT_ARGB_5551 RK_FORMAT_ABGR_5551 RK_FORMAT_RGB_888 RK_FORMAT_BGR_888 RK_FORMAT_RGB_565 RK_FORMAT_BGR_565 RK_FORMAT_YCbCr_420_SP RK_FORMAT_YCrCb_420_SP RK_FORMAT_YCbCr_422_SP RK_FORMAT_YCrCb_422_SP RK_FORMAT_YCbCr_444_SP RK_FORMAT_YCrCb_444_SP RK_FORMAT_YCbCr_420_P RK_FORMAT_YCrCb_420_P RK_FORMAT_YCbCr_422_P RK_FORMAT_YCrCb_422_P RK_FORMAT_YUYV_422 RK_FORMAT_YVYU_422 RK_FORMAT_UYVY_422 RK_FORMAT_VYUY_422 RK_FORMAT_YCbCr_400 RK_FORMAT_YCbCr_420_SP_10B RK_FORMAT_YCrCb_420_SP_10B RK_FORMAT_YCbCr_422_SP_10B RK_FORMAT_YCrCb_422_SP_10B RK_FORMA_A8 (only src for alpha blend) RK_FORMAT_BPP1 (only for color palette) RK_FORMAT_BPP2 (only for color palette) RK_FORMAT_BPP4 (only for color palette) RK_FORMAT_BPP8 (only for color palette)	RK_FORMAT_RGBA_8888 RK_FORMAT_BGRA_8888 RK_FORMAT_ARGB_8888 RK_FORMAT_ABGR_8888 RK_FORMAT_RGBX_8888 RK_FORMAT_BGRX_8888 RK_FORMAT_XRGB_8888 RK_FORMAT_XBGR_8888 RK_FORMAT_ARGB_4444 RK_FORMAT_ABGR_4444 RK_FORMAT_ARGB_5551 RK_FORMAT_ABGR_5551 RK_FORMAT_RGB_888 RK_FORMAT_BGR_888 RK_FORMAT_RGB_565 RK_FORMAT_BGR_565 RK_FORMAT_YCbCr_420_SP RK_FORMAT_YCrCb_420_SP RK_FORMAT_YCbCr_422_SP RK_FORMAT_YCrCb_422_SP RK_FORMAT_YCbCr_444_SP RK_FORMAT_YCrCb_444_SP RK_FORMAT_YCbCr_420_P RK_FORMAT_YCrCb_420_P RK_FORMAT_YCbCr_422_P RK_FORMAT_YCrCb_422_P RK_FORMAT_YUYV_422 RK_FORMAT_YVYU_422 RK_FORMAT_UYVY_422 RK_FORMAT_VYUY_422 RK_FORMAT_YCbCr_400 RK_FORMAT_Y4 (only RK3576) RK_FORMAT_Y8 (only RK3576)
	Swan	RV1126B		

RGA3	Orion	RK3588	RK_FORMAT_RGBA_8888 RK_FORMAT_BGRA_8888 RK_FORMAT_ARGB_8888 RK_FORMAT_ABGR_8888 RK_FORMAT_RGBX_8888 RK_FORMAT_BGRX_8888 RK_FORMAT_XRGB_8888 RK_FORMAT_XBGR_8888 RK_FORMAT_RGB_888 RK_FORMAT_BGR_888 RK_FORMAT_RGB_565 RK_FORMAT_BGR_565 RK_FORMAT_YCbCr_420_SP RK_FORMAT_YCrCb_420_SP RK_FORMAT_YCbCr_422_SP RK_FORMAT_YCrCb_422_SP RK_FORMAT_YUYV_422 RK_FORMAT_YVYU_422 RK_FORMAT_UYVY_422 RK_FORMAT_VYUY_422 RK_FORMAT_YCbCr_420_SP_10B RK_FORMAT_YCrCb_420_SP_10B RK_FORMAT_YCbCr_422_SP_10B RK_FORMAT_YCrCb_422_SP_10B	RK_FORMAT_RGBA_8888 RK_FORMAT_BGRA_8888 RK_FORMAT_RGBX_8888 RK_FORMAT_BGRX_8888 RK_FORMAT_RGB_888 RK_FORMAT_BGR_888 RK_FORMAT_RGB_565 RK_FORMAT_BGR_565 RK_FORMAT_YCbCr_420_SP RK_FORMAT_YCrCb_420_SP RK_FORMAT_YCbCr_422_SP RK_FORMAT_YCrCb_422_SP RK_FORMAT_YUYV_422 RK_FORMAT_YVYU_422 RK_FORMAT_UYVY_422 RK_FORMAT_VYUY_422 RK_FORMAT_YCbCr_420_SP_10B RK_FORMAT_YCrCb_420_SP_10B RK_FORMAT_YCbCr_422_SP_10B RK_FORMAT_YCrCb_422_SP_10B

> Note:
>
> 1). The "RK_FORMAT_YCbCr_400" format means that the YUV format only takes the Y channel, and is often used in 256 (2 to the 8th power) grayscale images. Here, it should be noted that since the YUV format exists in the RGB/YUV color space conversion, you need to pay attention to the color space configuration , for example, a full 256-level grayscale image needs to be configured as full range during conversion.
>
> 2). The "RK_FORMAT_Y4" format means that the YUV format only takes the Y channel and dithers to 4 bits. It is often used in 16 (2 to the 4th power) grayscale images. The precautions involved in the configuration of the color space conversion are the same as “RK_FORMAT_YCbCr_400”. "RK_FORMAT_Y8" is similar to "RK_FORMAT_Y4", also only 4 bits are valid, the difference is that only the high 4 bits are valid data, the low 4 bits are invalid data.

1.3 Image Format Alignment Instructions

Version	Byte_stride	Format	Alignment
RGA1 RGA1-Plus	4	RK_FORMAT_RGBA_8888 RK_FORMAT_BGRA_8888 RK_FORMAT_ARGB_8888 RK_FORMAT_ABGR_8888 RK_FORMAT_RGBX_8888 RK_FORMAT_BGRX_8888 RK_FORMAT_XRGB_8888 RK_FORMAT_XBGR_8888	width stride does not require alignment
		RK_FORMAT_RGBA_4444 RK_FORMAT_BGRA_4444 RK_FORMAT_ARGB_4444 RK_FORMAT_ABGR_4444 RK_FORMAT_RGBA_5551 RK_FORMAT_BGRA_5551 RK_FORMAT_ARGB_5551 RK_FORMAT_ABGR_5551 RK_FORMAT_RGB_565 RK_FORMAT_BGR_565	width stride must be 2-aligned
		RK_FORMAT_RGB_888 RK_FORMAT_BGR_888	width stride must be 4-aligned
		RK_FORMAT_YCbCr_420_SP RK_FORMAT_YCrCb_420_SP RK_FORMAT_YCbCr_422_SP RK_FORMAT_YCrCb_422_SP RK_FORMAT_YCbCr_420_P RK_FORMAT_YCrCb_420_P RK_FORMAT_YCbCr_422_P RK_FORMAT_YCrCb_422_P	width stride must be 4-aligned，x_offset、y_offset、width、height、height stride must be 2-aligned

RGA2 RGA2-Lite0 RGA2-Lite1 RGA2-Lite2 RGA2-Lite3 RGA2-Enhance RGA2-Pro	4	RK_FORMAT_RGBA_8888 RK_FORMAT_BGRA_8888 RK_FORMAT_ARGB_8888 RK_FORMAT_ABGR_8888 RK_FORMAT_RGBX_8888 RK_FORMAT_BGRX_8888 RK_FORMAT_XRGB_8888 RK_FORMAT_XBGR_8888	width stride does not require alignment
		RK_FORMAT_RGBA_4444 RK_FORMAT_BGRA_4444 RK_FORMAT_ARGB_4444 RK_FORMAT_ABGR_4444 RK_FORMAT_RGBA_5551 RK_FORMAT_BGRA_5551 RK_FORMAT_ARGB_5551 RK_FORMAT_ABGR_5551 RK_FORMAT_RGB_565 RK_FORMAT_BGR_565	width stride must be 2-aligned
		RK_FORMAT_YUYV_422 RK_FORMAT_YVYU_422 RK_FORMAT_UYVY_422 RK_FORMAT_VYUY_422 RK_FORMAT_YUYV_420 RK_FORMAT_YVYU_420 RK_FORMAT_UYVY_420 RK_FORMAT_VYUY_420	width stride must be 2-aligned，x_offset、y_offset、width、height、height stride must be 2-aligned
		RK_FORMAT_RGB_888 RK_FORMAT_BGR_888 RK_FORMAT_A8	width stride must be 4-aligned
		RK_FORMAT_YCbCr_420_SP RK_FORMAT_YCrCb_420_SP RK_FORMAT_YCbCr_422_SP RK_FORMAT_YCrCb_422_SP RK_FORMAT_YCbCr_444_SP RK_FORMAT_YCrCb_444_SP RK_FORMAT_YCbCr_420_P RK_FORMAT_YCrCb_420_P RK_FORMAT_YCbCr_422_P RK_FORMAT_YCrCb_422_P RK_FORMAT_YCbCr_400 RK_FORMAT_Y4 RK_FORMAT_Y8	width stride must be 4-aligned，x_offset、y_offset、width、height、height stride must be 2-aligned
		RK_FORMAT_YCbCr_420_SP_10B RK_FORMAT_YCrCb_420_SP_10B RK_FORMAT_YCbCr_422_SP_10B RK_FORMAT_YCrCb_422_SP_10B	width stride must be 16-aligned，x_offset、y_offset must be 64-aligned，width、height、height stride must be 2-aligned

RGA3	16	RK_FORMAT_RGBA_8888 RK_FORMAT_BGRA_8888 RK_FORMAT_ARGB_8888 RK_FORMAT_ABGR_8888 RK_FORMAT_RGBX_8888 RK_FORMAT_BGRX_8888 RK_FORMAT_XRGB_8888 RK_FORMAT_XBGR_8888	width stride must be 4-aligned
		RK_FORMAT_RGB_565 RK_FORMAT_BGR_565	width stride must be 8-aligned
		RK_FORMAT_YUYV_422 RK_FORMAT_YVYU_422 RK_FORMAT_UYVY_422 RK_FORMAT_VYUY_422	width stride must be 8-aligned，x_offset、y_offset、width、height、height stride must be 2-aligned
		RK_FORMAT_RGB_888 RK_FORMAT_BGR_888	width stride must be 16-aligned
		RK_FORMAT_YCbCr_420_SP RK_FORMAT_YCrCb_420_SP RK_FORMAT_YCbCr_422_SP RK_FORMAT_YCrCb_422_SP	width stride must be 16-aligned，x_offset、y_offset、width、height、height stride must be 2-aligned
		RK_FORMAT_YCbCr_420_SP_10B RK_FORMAT_YCrCb_420_SP_10B RK_FORMAT_YCbCr_422_SP_10B RK_FORMAT_YCrCb_422_SP_10B	width stride must be 64-aligned，x_offset、y_offset must be 4-aligned，width、height、height stride must be 2-aligned
		FBC mode	In addition to the format alignment requirements above，width stride、height stride must be 16-aligned
		TILE8*8 mode	In addition to the format alignment requirements above，width、height must be 8-aligned，input channel width stride、height stride must be 16-aligned。

> Note:
>
> 1). Alignment requirement formula: lcm(bpp，byte_stride * 8) / pixel_stride.
>
> 2). When loaded with multiple versions of hardware, chip platform constraints according to the most strict alignment requirements.

2. Version Description

RGA's support library librga.so updates the version number according to certain rules, indicating the submission of new features, compatibility, and bug fixes, and provides several ways to query the version number, so that developers can clearly determine whether the current library version is suitable for the current development environment when using librga.so. Detailed version update logs can be found in **CHANGLOG.md** in the root directory of source code.

2.1 librga API Version Description

2.1.1 Version Number Format and Update Rule

2.1.1.1 Version Number Format

```
major.minor.revision_[build]
```

example：

> 1.0.0_[0]

2.1.1.2 Update Rule

Name	Rule
major	Major version number, when submitting a version that is not backward compatible.
minor	Minor version number, when the functional API with backward compatibility is added.
revision	Revision version number, when submitting backward compatible feature additions or fatal bug fixes.
build	Compile version number, when backward compatibility issue is fixed.

2.1.2 Version Number Query

2.1.2.1 Query by Strings

Take Android R 64bit as example:

```
:/# strings vendor/lib64/librga.so |grep rga_api |grep version
rga_api version 1.0.0_[0]
```

2.1.2.2 Log Print

Version number is printed when each process first calls RGA API.

```
rockchiprga: rga_api version 1.0.0_[0]
```

2.1.2.3 Query by API

Call the following API to query the code version number, compilation version number, and RGA hardware version information. For details, see **API Description**.

```
querystring(RGA_VERSION);
```

> String format is as follows:
>
> RGA_api version : v1.0.0_[0]
> RGA version : RGA_2_Enhance

2.1.2.4 Query by Property

This method is supported only by Android system, and the property takes effect only after an existing process calls RGA.

```
:/# getprop |grep rga
[vendor.rga_api.version]: [1.0.0_[0]]
```

2.2 Driver Version Description

librga calls the RGA hardware based on the driver, it must be ensured that the driver version is within the supported range of the used librga library.

2.2.1 Version Number Format and Update Rule

2.2.1.1 Version Number Format

```
<driver_name>: v major.minor.revision
```

example：

> RGA2 Device Driver: v2.1.0
>
> RGA multicore Device Driver: v1.2.23

2.2.1.2 Update Rule

Name	Rule
major	Major version number, when submitting a version that is not backward compatible.
minor	Minor version number, when the functional API with backward compatibility is added.
revision	Revision version number, when submitting backward compatible feature additions or fatal bug fixes.

2.2.2 Version Number Query

2.2.2.1 Boot log query:

Use the following command to query the RGA driver initialization log after booting. Some early drivers do not print the version number, and this method is only applicable to some drivers.

```
dmesg |grep rga
```

example：

```
[    2.382393] rga3_core0 fdb60000.rga: Adding to iommu group 2
[    2.382651] rga: rga3_core0, irq = 33, match scheduler
[    2.383058] rga: rga3_core0 hardware loaded successfully, hw_version:3.0.76831.
[    2.383121] rga: rga3_core0 probe successfully
[    2.383687] rga3_core1 fdb70000.rga: Adding to iommu group 3
[    2.383917] rga: rga3_core1, irq = 34, match scheduler
[    2.384313] rga: rga3_core1 hardware loaded successfully, hw_version:3.0.76831.
[    2.384412] rga: rga3_core1 probe successfully
[    2.384893] rga: rga2, irq = 35, match scheduler
[    2.385238] rga: rga2 hardware loaded successfully, hw_version:3.2.63318.
[    2.385257] rga: rga2 probe successfully
[    2.385455] rga_iommu: IOMMU binding successfully, default mapping core[0x1]
[    2.385586] rga: Module initialized. v1.2.23
```

Among them, “v1.2.23” is the driver version number.

2.2.2.2 debug node query

The version number can be queried through the driver debugging node. If there is no following node, it means that the driver version that does not support query is currently running.

- Use the kernel with the CONFIG_ROCKCHIP_RGA_DEBUG_FS compile config enabled by default.

```
cat /sys/kernel/debug/rkrga/driver_version
```

- Use the kernel with the CONFIG_ROCKCHIP_RGA_PROC_FS compile config enabled.

```
cat /proc/rkrga/driver_version
```

example：

```
cat /sys/kernel/debug/rkrga/driver_version
RGA multicore Device Driver: v1.2.23
```

Here "RGA multicore Device Driver" means that the driver name is RGA multicore Device Driver, and "v1.2.23" means that the version is 1.2.23, which means that the current driver is the RGA multicore Device Driver(usually referred to as multi_rga driver) of version 1.2.23 .

```
cat /sys/kernel/debug/rkrga/driver_version
RGA2 Device Driver: v2.1.0
```

Here "RGA2 Device Driver" means that the driver name is RGA2 Device Driver, and "v2.1.0" means that the version number is 2.1.0, which means that the current driver is the RGA2 Device Driver (usually referred to as rga2 driver) of version 2.1.0.

2.3 Correspondence between versions

When using RGA, you need to confirm that the current operating environment can work normally. The following table shows the correspondence between commonly used librga and driver versions.

librga version	**driver version**	**hardware**
no version number	Drivers in the SDK	RGA1、RGA2
1.0.0 ~ 1.3.2	RGA Device Driver（kernel - 4.4 and above） RGA2 Device Driver（no version number or v2.1.0）	RGA1、RGA2
> 1.4.0	RGA multicore Device Driver（v1.2.0and above）	RGA2、RGA3
> 1.9.0	RGA Device Driver（kernel-4.4 and above） RGA2 Device Driver（no version number or v2.1.0） RGA multicore Device Driver（v1.2.0and above）	RGA1、RGA2、RGA3

3. API Description

RGA library librga.so realizes 2D graphics operations through the image buffer structure rga_info configuration. In order to obtain a better development experience, the common 2D image operation API is further encapsulated. The new API mainly contains the following features:

- API definitions refer to common 2D graphics API definitions in opencv/matlab, reducing the learning cost of secondary development.
- To eliminate compatibility problems caused by RGA hardware version differences, RGA query is added. Query mainly includes version information, large resolution and image format support.
- Add improcess API for 2D image compound operations. Compound operations are performed by passing in a set of predefined usage.
- Before performing image operation, the input and output image buffers need to be processed. The wrapbuffer_T API is called to pass the input and output image information to rga_buffer_t, which contains information such as resolution and image format.
- It supports to bind the image composite operation that cannot be completed in a single time as an RGA image task, and submit it to the driver and execute it one by one.

3.1 Overview

The software support library provides the following API, asynchronous mode only supports C++ implementation.

- **querystring**： Query information about the RGA hardware version and supported functions of chip platform, return a string.
- **imcheckHeader**: Verify the difference between the currently used header file version and the librga version.
- **importbuffer_T**： Import external buffer into RGA driver to achieve hardware fast access to discontinuous physical addresses (dma_fd, virtual address).
- **releasebuffer_handle**： Remove reference and map of the external buffer from inside the RGA driver.
- **wrapbuffer_handle** Quickly encapsulate the image buffer structure (rga_buffer_t）.
- **imbeginJob**：Create an RGA image processing job.
- **imendJob**： Submit and execute RGA image processing job.
- **imcancelJob**： Cancel and delete the RGA image processing job.
- **imcopy**： Call RGA for fast image copy.
- **imcopyTask**： Added fast image copy operation to RGA image job.
- **imresize**： Call RGA for fast image resize.
- **imresizeTask**： Added fast image resize operation to RGA image job.
- **impyramind**： Call RGA for fast image pyramid.
- **imcrop**： Call RGA for fast image cropping.
- **imcropTask**： Added fast image cropping operation to RGA image job.
- **imtranslate**： Call RGA for fast image translation.
- **imtranslateTask**： Added fast image translation operation to RGA image job.
- **imcvtcolor**： Call RGA for fast image format conversion.
- **imcvtcolorTask**： Added fast image format conversion operation to RGA image job.
- **imrotate**： Call RGA for fast image rotation.
- **imrotateTask**： Added fast image rotation operation to RGA image job.
- **imflip**： Call RGA for fast image flipping.
- **imflipTask**： Added fast image flipping operation to RGA image job.

- **imblend**: Call RGA for double channel fast image blending.
- **imblendTask**: Added double channel fast image blending operation to RGA image job.
- **imcomposite**: Call RGA for three-channel fast image composite.
- **imcompositeTask**: Added three-channel fast image composite operation to RGA image job.
- **imcolorkey**: Call RGA for fast image color key.
- **imcolorkeyTask**: Added fast image color key operation to RGA image job.
- **imosd**: Call RGA for fast image OSD.
- **imosdTask**: Added fast image OSD operation to RGA image job.
- **imquantize**: Call RGA for fast image operation point preprocessing (quantization).
- **imquantizeTask**: Added fast image operation point preprocessing (quantization) operation to RGA image job.
- **imrop**: Call RGA for fast image ROP.
- **imropTask**: Added fast image ROP operation to RGA image job.
- **imfill**: Call RGA for fast image filling.
- **imfillArray**: Call RGA to implement multiple groups of fast image filling.
- **imfillTask**: Added fast image filling operation to RGA image job.
- **imfillTaskArray**: Added multiple groups of fast image filling operation to RGA image job.
- **imrectangle**: Call RGA for fast drawing operation of equidistant rectangle border.
- **imrectangleArray**: Call RGA for multiple groups of fast drawing operation of equidistant rectangle border.
- **imrectangleTask**: Added fast drawing operation of equidistant rectangle border operation to RGA image job.
- **imrectangleTaskArray**: Added multiple groups of fast drawing operation of equidistant rectangle border operation to RGA image job.
- **immakeBorder**: Call RGA for fast drawing operation of the border.
- **immosaic**: call RGA for fast mosaic masking.
- **immosaicArray**: call RGA for fast multiple groups of mosaic masking.
- **immosaicTask**: Added fast mosaic masking operation to RGA image job.
- **immosaicTaskArray**: Added fast multiple groups of mosaic masking operation to RGA image job.
- **improcess**: Call RGA for fast image compound processing.
- **improcessTask**: Added fast image compound processing operation to RGA image job.
- **imcheck**: Verify whether the parameters are valid and whether the current hardware supports the operation.
- **imsync**: Synchronize task completion status in asynchronous mode.
- **imconfig**: Add default configuration to current thread context.

Support status of each API in different environments:

Item	API	Language	System	librga	RGA2 driver	Multi_RGA driver
common API	querystring	C++/C	Android / Linux / QNX / RT-thread	√	/	/
	imcheckHeader	C++	Android / Linux / QNX / RT-thread	⩾1.9.0	/	/
	imcheck	C++/C	Android / Linux / QNX / RT-thread	√	√	√
	imStrError	C++/C	Android / Linux / QNX / RT-thread	√	/	/
	imsync	C++/C	Android / Linux	⩾1.6.0	×	√
	imconfig	C++/C	Android / Linux	⩾1.6.0	×	√
buffer API	importbuffer_virtualaddr	C++/C	Android / Linux	⩾1.7.2	×	√
	importbuffer_physicaladdr	C++/C	Android / Linux	⩾1.7.2	×	√
	importbuffer_fd	C++/C	Android / Linux	⩾1.7.2	×	√
	releasebuffer_handle	C++/C	Android / Linux	⩾1.7.2	×	√
	wrapbuffer_handle	C++/C	Android / Linux	⩾1.7.2	/	/
	wrapbuffer_virtualaddr	C++/C	Android / Linux / QNX / RT-thread	√	/	/
	wrapbuffer_physicaladdr	C++/C	Android / Linux / QNX / RT-thread	√	/	/
	wrapbuffer_fd	C++/C	Android / Linux / QNX / RT-thread	√	/	/

Item	API	Language	System	librga	RGA2 driver	Multi_RGA driver
single process API	imcopy	C++/C	Android / Linux / QNX / RT-thread	√	√	√
	imresize	C++/C	Android / Linux / QNX / RT-thread	√	√	√
	impyramind	C++/C	Android / Linux / QNX / RT-thread	√	√	√
	imcrop	C++/C	Android / Linux / QNX / RT-thread	√	√	√
	imtranslate	C++/C	Android / Linux / QNX / RT-thread	√	√	√
	imcvtcolor	C++/C	Android / Linux / QNX / RT-thread	√	√	√
	imrotate	C++/C	Android / Linux / QNX / RT-thread	√	√	√
	imflip	C++/C	Android / Linux / QNX / RT-thread	√	√	√
	imblend	C++/C	Android / Linux / QNX / RT-thread	√	√	√
	imcomposite	C++/C	Android / Linux / QNX / RT-thread	√	√	√
	imcolorkey	C++/C	Android / Linux / QNX / RT-thread	√	√	√
	imosd	C++/C	Android / Linux / QNX / RT-thread	⩾1.8.0	×	√
	imquantize	C++/C	Android / Linux / QNX / RT-thread	√	√	√
	imrop	C++/C	Android / Linux / QNX / RT-thread	√	√	√
	imfill	C++/C	Android / Linux / QNX / RT-thread	√	√	√
	imfillArray	C++/C	Android / Linux	⩾1.9.0	×	√

Item	API	Language	System	librga	RGA2 driver	Multi_RGA driver
	imrectangle	C++/C	Android / Linux	⩾1.9.0	×	√
	imrectangleArray	C++/C	Android / Linux	⩾1.9.0	×	√
	immakeBorder	C++/C	Android / Linux	⩾1.9.0	×	√
	immosaic	C++/C	Android / Linux / QNX / RT-thread	⩾1.8.0	×	√
	immosaicArray	C++/C	Android / Linux	⩾1.9.0	×	√
	improcess	C++/C	Android / Linux / QNX / RT-thread	√	√	√

Item	API	Language	System	librga	RGA2 driver	Multi_RGA driver
task process API	imbeginJob	C++	Android / Linux	⩾1.9.0	×	⩾1.2.25
	imendJob	C++	Android / Linux	⩾1.9.0	×	⩾1.2.25
	imcancelJob	C++	Android / Linux	⩾1.9.0	×	⩾1.2.25
	imcopyTask	C++	Android / Linux	⩾1.9.0	×	⩾1.2.25
	imresizeTask	C++	Android / Linux	⩾1.9.0	×	⩾1.2.25
	imcropTask	C++	Android / Linux	⩾1.9.0	×	⩾1.2.25
	imtranslateTask	C++	Android / Linux	⩾1.9.0	×	⩾1.2.25
	imcvtcolorTask	C++	Android / Linux	⩾1.9.0	×	⩾1.2.25
	imrotateTask	C++	Android / Linux	⩾1.9.0	×	⩾1.2.25
	imflipTask	C++	Android / Linux	⩾1.9.0	×	⩾1.2.25
	imblendTask	C++	Android / Linux	⩾1.9.0	×	⩾1.2.25
	imcompositeTask	C++	Android / Linux	⩾1.9.0	×	⩾1.2.25
	imcolorkeyTask	C++	Android / Linux	⩾1.9.0	×	⩾1.2.25
	imosdTask	C++	Android / Linux	⩾1.9.0	×	⩾1.2.25
	imquantizeTask	C++	Android / Linux	⩾1.9.0	×	⩾1.2.25
	imropTask	C++	Android / Linux	⩾1.9.0	×	⩾1.2.25
	imfillTask	C++	Android / Linux	⩾1.9.0	×	⩾1.2.25
	imfillTaskArray	C++	Android / Linux	⩾1.9.0	×	⩾1.2.25
	imrectangleTask	C++	Android / Linux	⩾1.9.0	×	⩾1.2.25
	imrectangleTaskArray	C++	Android / Linux	⩾1.9.0	×	⩾1.2.25
	immosaicTask	C++	Android / Linux	⩾1.9.0	×	⩾1.2.25
	immosaicTaskArray	C++	Android / Linux	⩾1.9.0	×	⩾1.2.25

Item	API	Language	System	librga	RGA2 driver	Multi_RGA driver
	immosaicTask	C++	Android / Linux	⩾1.9.0	×	⩾1.2.25
	improcessTask	C++	Android / Linux	⩾1.9.0	×	⩾1.2.25
expand API	importbuffer_GraphicBuffer	C++	Android	√	/	/
	importbuffer_GraphicBuffer_handle	C++	Android	√	/	/
	importbuffer_AHardwareBuffer	C++	Android	√	/	/
	wrapbuffer_GraphicBuffer	C++	Android	√	/	/
	wrapbuffer_handle（GraphicBuffer handle）	C++	Android	√	/	/
	wrapbuffer_AHardwareBuffer	C++	Android	√	/	/

> Note:
>
> (√): Supported, (×): Not supported, (/): Not applicable.

3.2 Obtain RGA Version and Support Information

3.2.1 querystring

```
const char* querystring(int name);
```

> Query RGA basic information and resolution format.

Parameters	Description
name	RGA_VENDOR - vendor information RGA_VERSION - RGA version RGA_MAX_INPUT - max input resolution RGA_MAX_OUTPUT - max output resolution RGA_BYTE_STRIDE - stride alignment requirements RGA_SCALE_LIMIT - scale limit RGA_INPUT_FORMAT - input formats supported RGA_OUTPUT_FORMAT - output formats supported RGA_EXPECTED - expected performance RGA_ALL - print all information

Returns a string describing properties of RGA.

3.3 header version check

3.3.1 imcheckHeader

```
IM_API IM_STATUS imcheckHeader(im_api_version_t header_version = RGA_CURRENT_API_HEADER_VERSION);
```

> Verify the difference between the currently used header file version and the librga version.

Parameters	Description
header_version	Header file version, usually use the macro RGA_CURRENT_API_HEADER_VERSION.

Return IM_STATUS_SUCCESS on success or else negative error code.

3.4 Image Buffer Preprocessing

3.4.1 importbuffer_T

> For external memory that requires RGA processing, you can use importbuffer_T to map physical address of buffer to RGA driver and obtain the corresponding address of buffer, facilitating the subsequent stable and fast RGA call to complete the work.

Parameters(T)	Data Type	Description
virtual address	void *	image buffer virtual address
physical address	uint64_t	contiguous physical address of image buffer
fd	int	image buffer DMA file descriptor
GraphicBuffer handle	buffer_handle_t	image buffer handle, containing buffer address, file descriptor, resolution and format
GraphicBuffer	GraphicBuffer	android graphic buffer
AHardwareBuffer	AHardwareBuffer	chunks of memory that can be accessed by various hardware components in the system. https://developer.android.com/ndk/reference/group/a-hardware-buffer

> Performance varies when different buffer types call RGA, and the performance order is shown below:
>
> physical address > fd > virtual address
>
> The recommended buffer type is fd.

```
IM_API rga_buffer_handle_t importbuffer_fd(int fd, int size);
IM_API rga_buffer_handle_t importbuffer_virtualaddr(void *va, int size);
IM_API rga_buffer_handle_t importbuffer_physicaladdr(uint64_t pa, int size);
```

Parameter	Description
fd/va/pa	**[required]** external buffer
size	**[required]** memory size

```
IM_API rga_buffer_handle_t importbuffer_fd(int fd, int width, int height, int format);
IM_API rga_buffer_handle_t importbuffer_virtualaddr(void *va, int width, int height, int format);
IM_API rga_buffer_handle_t importbuffer_physicaladdr(uint64_t pa, int width, int height, int format);
```

Parameter	Description
fd/va/pa	**[required]** external buffer
width	**[required]** pixel width stride of the image buffer
height	**[required]** pixel height stride of the image buffer
format	**[required]** pixel format of the image buffer

```
IM_API rga_buffer_handle_t importbuffer_fd(int fd, im_handle_param_t *param);
IM_API rga_buffer_handle_t importbuffer_virtualaddr(void *va, im_handle_param_t *param);
IM_API rga_buffer_handle_t importbuffer_physicaladdr(uint64_t pa, im_handle_param_t *param);
```

Parameter	Description
fd/va/pa	**[required]** external buffer
param	**[required]** configure buffer parameters

```
IM_API rga_buffer_handle_t importbuffer_GraphicBuffer_handle(buffer_handle_t hnd);
IM_API rga_buffer_handle_t importbuffer_GraphicBuffer(sp<GraphicBuffer> buf);
IM_API rga_buffer_handle_t importbuffer_AHardwareBuffer(AHardwareBuffer *buf);
```

Parameter	Description
hnd/buf	**[required]** external buffer

Returns rga_buffer_handle_t to describe the memory handle.

3.4.2 releasebuffer_handle

> After finishing calling RGA using external memory, you need to call releasebuffer_handle through memory handle to remove the mapping and binding between buffer and RGA driver, and release the resource inside RGA driver.

```
IM_API IM_STATUS releasebuffer_handle(rga_buffer_handle_t handle);
```

Return IM_STATUS_SUCCESS on success or else negative error code.

3.4.3 wrapbuffer_handle

> In IM2D library API parameters, input image and output image should support multiple types, which mainly include memory, image format, image width and height. Before performing corresponding image operation, you need to call wrapbuffer_handle to convert the input and output image parameters into rga_buffer_t structure as the input parameter of user API.

```
rga_buffer_t wrapbuffer_handle(rga_buffer_handle_t handle,
                               int width,
                               int height,
                               int format,
                               int wstride = width,
                               int hstride = height);
```

Parameter	Description
handle	**[required]** RGA buffer handle
width	**[required]** pixel width of the image that needs to be processed
height	**[required]** pixel height of the image that needs to be processed
format	**[required]** pixel format
wtride	**[optional]** pixel width stride of the image
hstride	**[optional]** pixel width stride of the image

Returns a rga_buffer_t to desribe image information.

3.5 Image processing job create

3.5.1 imbeginJob

```
IM_API im_job_handle_t imbeginJob(uint64_t flags = 0);
```

> Create an RGA image processing job, which will return a job handle, job_handle can be used to add/remove RGA image operations, submit/execute the job.

Parameter	Description
flags	**[optional]** job flags

Returns im_job_handle_t to describe the job handle.

3.6 Image processing job submit

3.6.1 imendJob

```
IM_API IM_STATUS imendJob(im_job_handle_t job_handle,
                          int sync_mode = IM_SYNC,
                          int acquire_fence_fd = 0,
                          int *release_fence_fd = NULL);
```

> Submit and execute RGA image processing job. When the job is complete, the currently completed RGA image processing job resource is deleted.

Parameter	Description
job_handle	**[required]** job handle
sync_mode	**[optional]** wait until operation complete
acquire_fence_fd	**[optional]** Used in async mode, run the job after waiting foracquire_fence signal
release_fence_fd	**[optional]** Used in async mode, as a parameter of imsync()

Return IM_STATUS_SUCCESS on success or else negative error code.

3.7 Image processing job cancel

3.7.1 imcancelJob

```
IM_API IM_STATUS imcancelJob(im_job_handle_t job_handle);
```

> cancel and delete RGA image processing job.

Parameter	Description
job_handle	**[required]** job handle

Return IM_STATUS_SUCCESS on success or else negative error code.

3.8 Image Copy

3.8.1 imcopy

```
IM_STATUS imcopy(const rga_buffer_t src,
                 rga_buffer_t dst,
                 int sync = 1),
                 int *release_fence_fd = NULL);
```

> Copy the image, RGA basic operation. Its function is similar to memcpy.

Parameter	Description
src	**[required]** input image
dst	**[required]** output image
sync	**[optional]** wait until operation complete
release_fence_fd	**[optional]**Used in async mode, as a parameter of imsync()

Return IM_STATUS_SUCCESS on success or else negative error code.

3.8.2 imcopyTask

```
IM_API IM_STATUS imcopyTask(im_job_handle_t job_handle,
                            const rga_buffer_t src,
                            rga_buffer_t dst);
```

> Add an image copy operation to the specified job through job_handle. The configuration parameters are the same as imcopy.

Parameter	Description
job_handle	**[required]** job handle
src	**[required]** input image
dst	**[required]** output image

Return IM_STATUS_SUCCESS on success or else negative error code.

3.9 Image Resizing and Image Pyramid

3.9.1 imresize

```
IM_STATUS
imresize(const rga_buffer_t src,
         rga_buffer_t dst,
         double fx = 0,
         double fy = 0,
         int interpolation = INTER_LINEAR,
         int sync = 1,
         int *release_fence_fd = NULL);
```

> According to different scenario, you can choose to configure dst to describe the output image size of resizing, or configure the scale factor fx/fy to resize at a specified ratio. When dst and fx/fy are configured at the same time, the calculated result of fx/fy is used as the output image size.
>
> Only hardware version RGA1/RGA1 plus supports interpolation configuration.
>
> Note: When resizing with fx/fy, format such as YUV that requires width and height alignment will force downward alignment to meet the requirements. Using this feature may affect the expected resizing effect.

Parameters	Description
src	**[required]** input image
dst	**[required]** output image; it has the size dsize (when it is non-zero) or the size computed from src.size(), fx, and fy; the type of dst is the same as of src.
fx	**[optional]** scale factor along the horizontal axis; when it equals 0, it is computed as: fx = (double) dst.width / src.width
fy	**[optional]** scale factor along the vertical axis; when it equals 0, it is computed as: fy = (double) dst.height / src.height
interpolation	**[optional]** interpolation method: INTER_NEAREST - a nearest-neighbor interpolation INTER_LINEAR - a bilinear interpolation (used by default) INTER_CUBIC - a bicubic interpolation over 4x4 pixel neighborhood
sync	**[optional]** wait until operation complete
release_fence_fd	**[optional]**Used in async mode, as a parameter of imsync()

Return IM_STATUS_SUCCESS on success or else negative error code.

3.9.2 impyramid

```
IM_STATUS impyramid (const rga_buffer_t src,
                     rga_buffer_t dst,
                     IM_SCALE direction)
```

> Pyramid scaling. Scale by 1/2 or twice, depending on the direction width and height.

Parameters	Description
src	**[required]** input image
dst	**[required]** output image;
direction	**[required]** scale mode IM_UP_SCALE —— up scale IM_DOWN_SCALE —— down scale
release_fence_fd	**[optional]**Used in async mode, as a parameter of imsync()

Return IM_STATUS_SUCCESS on success or else negative error code.

3.9.3 imresizeTask

```
IM_API IM_STATUS imresizeTask(im_job_handle_t job_handle,
                              const rga_buffer_t src,
                              rga_buffer_t dst,
                              double fx = 0,
                              double fy = 0,
                              int interpolation = 0);
```

> Add an image resize operation to the specified job through job_handle. The configuration parameters are the same as imresize.

Parameters	Description
job_handle	**[required]** job handle
src	**[required]** input image
dst	**[required]** output image; it has the size dsize (when it is non-zero) or the size computed from src.size(), fx, and fy; the type of dst is the same as of src.
fx	**[optional]** scale factor along the horizontal axis; when it equals 0, it is computed as: fx = (double) dst.width / src.width
fy	**[optional]** scale factor along the vertical axis; when it equals 0, it is computed as: fy = (double) dst.height / src.height
interpolation	**[optional]** interpolation method: INTER_NEAREST - a nearest-neighbor interpolation INTER_LINEAR - a bilinear interpolation (used by default) INTER_CUBIC - a bicubic interpolation over 4x4 pixel neighborhood

Return IM_STATUS_SUCCESS on success or else negative error code.

3.10 Image Cropping

3.10.1 imcrop

```
IM_STATUS imcrop(const rga_buffer_t src,
                 rga_buffer_t dst,
                 im_rect rect,
                 int sync = 1,
                 int *release_fence_fd = NULL);
```

> Perform image clipping by specifying the size of the region.

Parameter	Description
src	**[required]** input image
dst	**[required]** output image
rect	**[required]** crop region x - upper-left x coordinate y - upper-left y coordinate width - region width height - region height
sync	**[optional]** wait until operation complete
release_fence_fd	**[optional]**Used in async mode, as a parameter of imsync()

Return IM_STATUS_SUCCESS on success or else negative error code.

3.10.2 imcropTask

```
IM_API IM_STATUS imcropTask(im_job_handle_t job_handle,
                            const rga_buffer_t src,
                            rga_buffer_t dst,
                            im_rect rect);
```

> Add an image crop operation to the specified job through job_handle. The configuration parameters are the same as imcrop.

Parameter	Description
job_handle	**[required]** job handle
src	**[required]** input image
dst	**[required]** output image
rect	**[required]** crop region x - upper-left x coordinate y - upper-left y coordinate width - region width height - region height

Return IM_STATUS_SUCCESS on success or else negative error code.

3.11 Image Translation

3.11.1 imtranslate

```
IM_STATUS imtranslate(const rga_buffer_t src,
                      rga_buffer_t dst,
                      int x,
                      int y,
                      int sync = 1,
                      int *release_fence_fd = NULL);
```

> Image translation. Move to (x, y) position, the width and height of src and dst must be the same, the excess part will be clipped.

Parameter	Description
src	**[required]**input image
dst	**[required]** output image
x	**[optional]** horizontal translation
y	**[optional]** vertical translation
sync	**[optional]** wait until operation complete
release_fence_fd	**[optional]**Used in async mode, as a parameter of imsync()

Return IM_STATUS_SUCCESS on success or else negative error code.

3.11.2 imtranslateTask

```
IM_API IM_STATUS imtranslateTask(im_job_handle_t job_handle,
                                 const rga_buffer_t src,
                                 rga_buffer_t dst,
                                 int x,
                                 int y);
```

> Add an image translation operation to the specified job through job_handle. The configuration parameters are the same as imtranslate.

Parameter	Description
job_handle	**[required]** job handle
src	**[required]**input image
dst	**[required]** output image
x	**[required]** horizontal translation
y	**[required]** vertical translation

Return IM_STATUS_SUCCESS on success or else negative error code.

3.12 Image Format Conversion

3.12.1 imcvtcolor

```
IM_STATUS imcvtcolor(rga_buffer_t src,
                     rga_buffer_t dst,
                     int sfmt,
                     int dfmt,
                     int mode = IM_COLOR_SPACE_DEFAULT,
                     int sync = 1,
                     int *release_fence_fd = NULL);
```

> Image format conversion，specific format support varies according to soc, please refer to the **Image Format Supported** section.

> The format can be set by rga_buffer_t, or configure the input image and output image formats respectively by sfmt/dfmt. When it comes to YUV/RGB color gamut conversion, you can configure the converted color gamut through mode, and the conversion is performed according to the BT.601 limit range by default.

parameter	Description
src	**[required]** input image
dst	**[required]** output image
sfmt	**[optional]** source image format
dfmt	**[optional]** destination image format
Mode	**[optional]** color space mode: IM_YUV_TO_RGB_BT601_LIMIT IM_YUV_TO_RGB_BT601_FULL IM_YUV_TO_RGB_BT709_LIMIT IM_RGB_TO_YUV_BT601_LIMIT IM_RGB_TO_YUV_BT601_FULL IM_RGB_TO_YUV_BT709_LIMIT
sync	**[optional]** wait until operation complete
release_fence_fd	**[optional]**Used in async mode, as a parameter of imsync()

Return IM_STATUS_SUCCESS on success or else negative error code.

3.12.2 imcvtcolorTask

```
IM_API IM_STATUS imcvtcolorTask(im_job_handle_t job_handle,
                                rga_buffer_t src,
                                rga_buffer_t dst,
                                int sfmt,
                                int dfmt,
                                int mode = IM_COLOR_SPACE_DEFAULT);
```

> Add an image format conversion operation to the specified job through job_handle. The configuration parameters are the same as imcvtcolor.

parameter	Description
job_handle	**[required]** job handle
src	**[required]** input image
dst	**[required]** output image
sfmt	**[optional]** source image format
dfmt	**[optional]** destination image format
Mode	**[optional]** color space mode: IM_YUV_TO_RGB_BT601_LIMIT IM_YUV_TO_RGB_BT601_FULL IM_YUV_TO_RGB_BT709_LIMIT IM_RGB_TO_YUV_BT601_LIMIT IM_RGB_TO_YUV_BT601_FULL IM_RGB_TO_YUV_BT709_LIMIT

Return IM_STATUS_SUCCESS on success or else negative error code.

3.13 Image Rotation

3.13.1 imrotate

```
IM_STATUS imrotate(const rga_buffer_t src,
                   rga_buffer_t dst,
                   int rotation,
                   int sync = 1,
                   int *release_fence_fd = NULL);
```

> Support image rotation 90,180,270 degrees.

Parameter	Description
src	**[required]** input image
dst	**[required]** output image
rotation	**[required]** rotation angle: 0 IM_HAL_TRANSFORM_ROT_90 IM_HAL_TRANSFORM_ROT_180 IM_HAL_TRANSFORM_ROT_270
sync	**[optional]** wait until operation complete
release_fence_fd	**[optional]**Used in async mode, as a parameter of imsync()

Return IM_STATUS_SUCCESS on success or else negative error code.

3.13.2 imrotateTask

```
IM_API IM_STATUS imrotateTask(im_job_handle_t job_handle,
                              const rga_buffer_t src,
                              rga_buffer_t dst,
                              int rotation);
```

> Add an image rotation operation to the specified job through job_handle. The configuration parameters are the same as imrotate.

Parameter	Description
job_handle	**[required]** job handle
src	**[required]** input image
dst	**[required]** output image
rotation	**[required]** rotation angle: IM_HAL_TRANSFORM_ROT_90 IM_HAL_TRANSFORM_ROT_180 IM_HAL_TRANSFORM_ROT_270

Return IM_STATUS_SUCCESS on success or else negative error code.

3.14 Image Mirror Flip

3.14.1 imfilp

```
IM_STATUS imflip (const rga_buffer_t src,
                  rga_buffer_t dst,
                  int mode,
                  int sync = 1,
                  int *release_fence_fd = NULL);
```

> Support image to do horizontal, vertical mirror flip.

Parameter	Description
src	**[required]** input image
dst	**[required]** output image
mode	**[optional]** flip mode: 0 IM_HAL_TRANSFORM_FLIP_H IM_HAL_TRANSFORM_FLIP_V
sync	**[optional]** wait until operation complete
release_fence_fd	**[optional]**Used in async mode, as a parameter of imsync()

Return IM_STATUS_SUCCESS on success or else negative error code.

3.14.2 imflipTask

```
IM_API IM_STATUS imflipTask(im_job_handle_t job_handle,
                            const rga_buffer_t src,
                            rga_buffer_t dst,
                            int mode);
```

> Add an image flip operation to the specified job through job_handle. The configuration parameters are the same as imflip.

Parameter	Description
job_handle	**[required]** job handle
src	**[required]** input image
dst	**[required]** output image
mode	**[required]** flip mode: IM_HAL_TRANSFORM_FLIP_H_V IM_HAL_TRANSFORM_FLIP_H IM_HAL_TRANSFORM_FLIP_V

Return IM_STATUS_SUCCESS on success or else negative error code.

3.15 Image Blending

3.15.1 imblend/imcomposite

```
IM_STATUS imblend(const rga_buffer_t srcA,
                  rga_buffer_t dst,
                  int mode = IM_ALPHA_BLEND_SRC_OVER,
                  int sync = 1,
                  int *release_fence_fd = NULL);
```

> RGA uses A+B -> B image two-channel blending mode to perform Alpha superposition for foreground image (srcA channel) and background image (dst channel) according to the configured blending model, and output the blending result to dst channel. When no mode is configured, it is set to src-over mode by default.

```
IM_STATUS imcomposite(const rga_buffer_t srcA,
                      const rga_buffer_t srcB,
                      rga_buffer_t dst,
                      int mode = IM_ALPHA_BLEND_SRC_OVER,
                      int sync = 1,
                      int *release_fence_fd = NULL);
```

> RGA uses A+B -> C image three-channel blending mode to perform Alpha superposition for foreground image (srcA channel) and background image (srcB channel) according to the configured blending model, and output the blending result to dst channel.

mode in the two image blending modes can be configured with different **Porter-Duff blending model**:

> Before describing the Porter-Duff blending model, give the following definitions:
>
> - S -**Marks the source image in two blended images**，namely the foreground image, short for source.
> - D -**Marks the destination image in two blended images**，namely the background image, short for destination.
> - R -**Marks the result of two images blending**，short for result.
> - c -**Marks the color of the pixel**，the RGB part of RGBA, which describes the color of the image itself, short for color.（**Note**，Color values (RGB) in the Porter-Duff blending model are all left-multiplied results, that is, the product of original color and transparency. If the color values are not left-multiplied, pre-multiplied operations（Xc = Xc * Xa） are required.）
> - a -**Marks the Alpha of the pixel**，Namely the A part of RGBA, describe the transparency of the image itself, short for Alpha.
> - f -**Marks factors acting on C or A**，short for factor.
>
> The core formula of Porter-Duff blending model is as follows:
>
> Rc = Sc * Sf + Dc * Df;
>
> that is： result color = source color * source factor + destination color * destination factor.
>
> Ra = Sa * Sf + Da * Df;
>
> that is： result Alpha = source Alpha * source factor + destination Alpha * destination factor.

RGA supports following blending models:

> SRC:
>
> Sf = 1， Df = 0；
>
> [Rc，Ra] = [Sc，Sa]；
>
> DST:
>
> Sf = 0， Df = 1；
>
> [Rc，Ra] = [Dc，Da]；
>
> SRC_OVER：

> Sf = 1， Df = （1 - Sa）;
>
> [Rc，Ra] = [Sc + (1 - Sa) * Dc， Sa + (1 - Sa) * Da];
>
> DST_OVER:
>
> Sf = (1 - Da)， Df = 1;
>
> [Rc，Ra] = [Sc * (1 - Da) + Dc， Sa * (1 - Da) + Da] ;

【Note】Image blending model does not support the YUV format image blending, the YUV format is not support in dst image of imblend , the YUV format is not support in srcB image of imcomposite.

Parameter	Description
srcA	**[required]** input image A
srcB	**[required]** input image B
dst	**[required]** output image
mode	**[optional]** blending mode: IM_ALPHA_BLEND_SRC —— SRC mode IM_ALPHA_BLEND_DST —— DST mode IM_ALPHA_BLEND_SRC_OVER —— SRC OVER mode IM_ALPHA_BLEND_DST_OVER —— DST OVER mode IM_ALPHA_BLEND_PRE_MUL —— Enable premultipling. When premultipling is required, this identifier must be processed with other mode identifiers, then assign to mode
sync	**[optional]** wait until operation complete
release_fence_fd	**[optional]**Used in async mode, as a parameter of imsync()

Return IM_STATUS_SUCCESS on success or else negative error code.

3.15.2 imblendTask/imcompositeTask

```
IM_API IM_STATUS imblendTask(im_job_handle_t job_handle,
                             const rga_buffer_t fg_image,
                             rga_buffer_t bg_image,
                             int mode = IM_ALPHA_BLEND_SRC_OVER);
```

> Add an A+B -> B image two-channel blending operation to the specified job through job_handle. The configuration parameters are the same as imblend. When no mode is configured, it is set to src-over mode by default.

```
IM_API IM_STATUS imcompositeTask(im_job_handle_t job_handle,
                                 const rga_buffer_t fg_image,
                                 const rga_buffer_t bg_image,
                                 rga_buffer_t output_image,
                                 int mode = IM_ALPHA_BLEND_SRC_OVER);
```

> Add an A+B -> C image three-channel blending operation to the specified job through job_handle. The configuration parameters are the same as imcomposite.

【Note】Image blending model does not support the YUV format image blending, the YUV format is not support in dst image of imblend , the YUV format is not support in srcB image of imcomposite.

Parameter	Description
job_handle	**[required]** job handle
fg_image	**[required]** foreground image
bg_image	**[required]** background image, when A+B->B it is also the output destination image.
output_image	**[required]** output destination image.
mode	**[optional]** blending mode: IM_ALPHA_BLEND_SRC —— SRC mode IM_ALPHA_BLEND_DST —— DST mode IM_ALPHA_BLEND_SRC_OVER —— SRC OVER mode IM_ALPHA_BLEND_DST_OVER —— DST OVER mode IM_ALPHA_BLEND_PRE_MUL —— Enable premultipling. When premultipling is required, this identifier must be processed with other mode identifiers, then assign to mode

Return IM_STATUS_SUCCESS on success or else negative error code.

3.16 Color Key

3.16.1 imcolorkey

```
IM_STATUS imcolorkey(const rga_buffer_t src,
                     rga_buffer_t dst,
                     im_colorkey_range range,
                     int mode = IM_ALPHA_COLORKEY_NORMAL,
                     int sync = 1,
                     int *release_fence_fd = NULL);
```

> Color Key is to preprocesses the source image, zeros the alpha component of pixels that meet the Color Key filtering conditions, wherein the Color Key filtering conditions are non-transparent color values, and performs the alpha blending mode between the preprocessed source image and the destination image.
>
> This mode only supports the Color Key operation on the source image (src) region of the image for the set Color range, and overlays on the destination image (dst) region.

> IM_ALPHA_COLORKEY_NORMAL is the normal mode, that is, the colors in the set color range are used as the filtering condition. The Alpha components of pixels in this color range are set zero; IM_ALPHA_COLORKEY_INVERTED is inverted. When no mode is configured, it is set to IM_ALPHA_COLORKEY_NORMAL mode by default.

Parameters	Range	Description
max	0x0 ~ 0xFFFFFFFF	The max color range to cancel/scratch, arranged as ABGR
min	0x0 ~ 0xFFFFFFFF	The min color range to cancel/scratch, arranged as ABGR

parameter	Description
src	**[required]** input image
dst	**[required]** output image
range	**[required]** Target color range typedef struct im_colorkey_range { int max; int min; } im_colorkey_value;
Mode	**[required]** Color Key mode： IM_ALPHA_COLORKEY_NORMAL IM_ALPHA_COLORKEY_INVERTED
sync	**[optional]** wait until operation complete
release_fence_fd	**[optional]**Used in async mode, as a parameter of imsync()

Return IM_STATUS_SUCCESS on success or else negative error code.

3.16.2 imcolorkeyTask

```
IM_API IM_STATUS imcolorkeyTask(im_job_handle_t job_handle,
                                const rga_buffer_t fg_image,
                                rga_buffer_t bg_image,
                                im_colorkey_range range,
                                int mode = IM_ALPHA_COLORKEY_NORMAL);
```

> Add an image Color Key operation to the specified job through job_handle. The configuration parameters are the same as imcolorkey.

Parameters	Range	Description
max	0x0 ~ 0xFFFFFFFF	The max color range to cancel/scratch, arranged as ABGR
min	0x0 ~ 0xFFFFFFFF	The min color range to cancel/scratch, arranged as ABGR

parameter	Description
job_handle	**[required]** job handle
src	**[required]** input image
dst	**[required]** output image
range	**[required]** Target color range typedef struct im_colorkey_range { int max; int min; } im_colorkey_value;
Mode	**[required]** Color Key mode： IM_ALPHA_COLORKEY_NORMAL IM_ALPHA_COLORKEY_INVERTED

Return IM_STATUS_SUCCESS on success or else negative error code.

3.17 OSD

3.17.1 imosd

```
IM_API IM_STATUS imosd(const rga_buffer_t osd,
                       const rga_buffer_t bg_image,
                       const im_rect osd_rect,
                       im_osd_t *osd_config,
                       int sync = 1,
                       int *release_fence_fd = NULL);
```

> OSD (On-Screen-Display), can superimpose text information on video pictures, and perform brightness statistics and automatic color inversion functions for fonts.

parameter	Description
OSD	**[required]** osd block image
bg_image	**[required]** output image
osd_rect	**[required]** image region to OSD
osd_config	**[required]** OSD function config
sync	**[optional]** wait until operation complete
release_fence_fd	**[optional]**Used in async mode, as a parameter of imsync()

Return IM_STATUS_SUCCESS on success or else negative error code.

3.17.2 imosdTask

```
IM_API IM_STATUS imosdTask(im_job_handle_t job_handle,
                           const rga_buffer_t osd,
                           const rga_buffer_t bg_image,
                           const im_rect osd_rect,
                           im_osd_t *osd_config);
```

> Add an OSD operation to the specified job through job_handle. The configuration parameters are the same as imosd.

parameter	Description
job_handle	**[required]** job handle
OSD	**[required]** osd block image
dst	**[required]** output image
osd_rect	**[required]** image region to OSD
osd_config	**[required]** OSD function config

Return IM_STATUS_SUCCESS on success or else negative error code.

3.18 Pre-processing of NN Operation Points (Quantization)

3.18.1 imquantize

```
IM_STATUS imquantize(const rga_buffer_t src,
                     rga_buffer_t dst,
                     rga_nn_t nn_info,
                     int sync = 1,
                     int *release_fence_fd = NULL);
```

> Currently supported only on RV1126 / RV1109. NN operation point pre-processing, three channels of RGB of image can be separately configured with offset and scale.

Formula:

```
dst = 【(src + offset) * scale 】
```

Parameters range:

Parameters	Range	Description
scale	0 ~ 3.99	10bit，From left to right, the highest 2 bits indicate the integer part and the lowest 8 bits indicate the decimal part
offset	-255 ~ 255	9bit，From left to right, the high indicates the sign bit, and the low indicates the offset from 0 to 255

Parameter	Description
src	**[required]** input image
dst	**[required]** output image
nn_info	**[required]** rga_nn_t struct configures the offset and scale of the three RGB channels respectively typedef struct rga_nn { int nn_flag; int scale_r; int scale_g; int scale_b; int offset_r; int offset_g; int offset_b; } rga_nn_t;
sync	**[optional]** wait until operation complete
release_fence_fd	**[optional]**Used in async mode, as a parameter of imsync()

Return IM_STATUS_SUCCESS on success or else negative error code.

3.18.2 imquantizeTask

```
IM_API IM_STATUS imquantizeTask(im_job_handle_t job_handle,
                                const rga_buffer_t src,
                                rga_buffer_t dst,
                                im_nn_t nn_info);
```

> Add an image quantization conversion operation to the specified job through job_handle. The configuration parameters are the same as imquantize.

parameter	Description
job_handle	**[required]** job handle
src	**[required]** input image
dst	**[required]** output image
nn_info	**[required]** rga_nn_t结构体对RGB三个通道offset及scale进行单独配置 typedef struct rga_nn { int nn_flag; int scale_r; int scale_g; int scale_b; int offset_r; int offset_g; int offset_b; } rga_nn_t;

Return IM_STATUS_SUCCESS on success or else negative error code.

3.19 Image ROP

3.19.1 imrop

```
IM_STATUS imrop(const rga_buffer_t src,
                rga_buffer_t dst,
                int rop_code,
                int sync = 1,
                int *release_fence_fd = NULL);
```

> Perform ROP, and, or, not operations on two images

Parameter	Description
src	**[required]** input image
dst	**[required]** output image
rop_code	**[required]** rop code mode IM_ROP_AND : dst = dst **AND** src; IM_ROP_OR : dst = dst **OR** src IM_ROP_NOT_DST : dst = **NOT** dst IM_ROP_NOT_SRC : dst = **NOT** src IM_ROP_XOR : dst = dst **XOR** src IM_ROP_NOT_XOR : dst = **NOT** (dst **XOR** src)
sync	**[optional]** wait until operation complete
release_fence_fd	**[optional]**Used in async mode, as a parameter of imsync()

Return IM_STATUS_SUCCESS on success or else negative error code

3.19.2 imropTask

```
IM_API IM_STATUS imropTask(im_job_handle_t job_handle,
                           const rga_buffer_t src,
                           rga_buffer_t dst,
                           int rop_code);
```

> Add an image ROP conversion operation to the specified job through job_handle. The configuration parameters are the same as imrop.

parameter	Description
job_handle	**[required]** job handle
src	**[required]** input image
dst	**[required]** output image
rop_code	**[required]** rop code mode IM_ROP_AND : dst = dst **AND** src; IM_ROP_OR : dst = dst **OR** src IM_ROP_NOT_DST : dst = **NOT** dst IM_ROP_NOT_SRC : dst = **NOT** src IM_ROP_XOR : dst = dst **XOR** src IM_ROP_NOT_XOR : dst = **NOT** (dst **XOR** src)

Return IM_STATUS_SUCCESS on success or else negative error code.

3.20 Image Color Filling

3.20.1 imfill

```
IM_STATUS imfill(rga_buffer_t buf,
                 im_rect rect,
                 int color = 0x00000000,
                 int sync = 1);
```

> Color fills the specified area rect of the image.
>
> Color parameters from high to low are respectively R，G，B，A. For example, red: color = 0xff000000.

【Note】 The width and height of the rect must be greater than or equal to 2

Parameter	Description
src	**[required]** input image
dst	**[required]** output image
rect	**[required]** image region to fill specified color width and height of rect must be greater than or equal to 2
color	**[required]** fill with color, default=0x00000000
sync	**[optional]** wait until operation complete
release_fence_fd	**[optional]**Used in async mode, as a parameter of imsync()

Return IM_STATUS_SUCCESS on success or else negative error code.

3.20.2 imfillArray

```
IM_API IM_STATUS imfillArray(rga_buffer_t dst,
                             im_rect *rect_array,
                             int array_size,
                             uint32_t color,
                             int sync = 1,
                             int *release_fence_fd = NULL);
```

> Color fills multiple areas of the image one by one.
>
> Color parameters from high to low are respectively R，G，B，A. For example, red: color = 0xff000000.

【Note】 The width and height of the rect must be greater than or equal to 2

Parameter	Description
dst	**[required]** target image
rect_array	**[required]** image region array_ptr to fill specified color width and height of rect must be greater than or equal to 2
array_size	**[required]** size of region arrays.
color	**[required]** fill with color
sync	**[optional]** wait until operation complete
release_fence_fd	**[optional]**Used in async mode, as a parameter of imsync()

Return IM_STATUS_SUCCESS on success or else negative error code.

3.20.3 imfillTask

```
IM_API IM_STATUS imfillTask(im_job_handle_t job_handle,
                            rga_buffer_t dst,
                            im_rect rect,
                            uint32_t color);
```

> Add an image color fill operation to the specified job through job_handle. The configuration parameters are the same as imfill.

【Note】 The width and height of the rect must be greater than or equal to 2

Parameter	Description
job_handle	**[required]** job handle
dst	**[required]** target image
rect	**[required]** image region to fill specified color width and height of rect must be greater than or equal to 2
color	**[required]** fill with color

Return IM_STATUS_SUCCESS on success or else negative error code.

3.20.4 imfillTaskArray

```
IM_API IM_STATUS imfillTaskArray(im_job_handle_t job_handle,
                                 rga_buffer_t dst,
                                 im_rect *rect_array,
                                 int array_size,
                                 uint32_t color);
```

> Add an image color fill multiple areas operation to the specified job through job_handle. The configuration parameters are the same as imfillArray.

【Note】 The width and height of the rect must be greater than or equal to 2

Parameter	Description
job_handle	**[required]** job handle
dst	**[required]** target image
rect_array	**[required]** image region array_ptr to fill specified color width and height of rect must be greater than or equal to 2
array_size	**[required]** size of region arrays.
color	**[required]** fill with color

Return IM_STATUS_SUCCESS on success or else negative error code.

3.20.5 imrectangle

```
IM_API IM_STATUS imrectangle(rga_buffer_t dst,
                             im_rect rect,
                             uint32_t color,
                             int thickness,
                             int sync = 1,
                             int *release_fence_fd = NULL);
```

> Draw a border with a thickness of "thickness" to the specified area rect of the image (described as the outer diameter of the border) according to the specified color by "color", and fill a solid rectangle when the thickness is negative.
>
> Color parameters from high to low are respectively R，G，B，A. For example, red: color = 0xff000000.

【Note】 The width and height of the rect must be greater than or equal to 2

Parameter	Description
dst	**[required]** target image
rect	**[required]** image region to fill specified color width and height of rect must be greater than or equal to 2
color	**[required]** fill with color
thickness	**[required]** Thickness of lines that make up the rectangle. Negative values, like -1, mean that the function has to draw a filled rectangle.
sync	**[optional]** wait until operation complete
release_fence_fd	**[optional]**Used in async mode, as a parameter of imsync()

Return IM_STATUS_SUCCESS on success or else negative error code.

3.20.6 imrectangleArray

```
IM_API IM_STATUS imrectangleArray(rga_buffer_t dst,
                                  im_rect *rect_array,
                                  int array_size,
                                  uint32_t color,
                                  int thickness,
                                  int sync = 1,
                                  int *release_fence_fd = NULL);
```

> Draw multiple border with a thickness of "thickness" to the specified area rect of the image (described as the outer diameter of the border) according to the specified color by "color", and fill a solid rectangle when the thickness is negative.
>
> Color parameters from high to low are respectively R，G，B，A. For example, red: color = 0xff000000.

【Note】 The width and height of the rect must be greater than or equal to 2

Parameter	Description
dst	**[required]** target image
rect_array	**[required]** image region array_ptr to fill specified color width and height of rect must be greater than or equal to 2
array_size	**[required]** size of region arrays.
color	**[required]** fill with color
thickness	**[required]** Thickness of lines that make up the rectangle. Negative values, like -1, mean that the function has to draw a filled rectangle.
sync	**[optional]** wait until operation complete
release_fence_fd	**[optional]**Used in async mode, as a parameter of imsync()

Return IM_STATUS_SUCCESS on success or else negative error code.

3.20.7 imrectangleTask

```
IM_API IM_STATUS imrectangleTask(im_job_handle_t job_handle,
                                 rga_buffer_t dst,
                                 im_rect rect,
                                 uint32_t color,
                                 int thickness);
```

> Add an Draw border operation to the specified job through job_handle. The configuration parameters are the same as imrectangle.

【Note】 The width and height of the rect must be greater than or equal to 2

Parameter	Description
job_handle	**[required]** job handle
dst	**[required]** target image
rect	**[required]** image region to fill specified color width and height of rect must be greater than or equal to 2
color	**[required]** fill with color
thickness	**[required]** Thickness of lines that make up the rectangle. Negative values, like -1, mean that the function has to draw a filled rectangle.

Return IM_STATUS_SUCCESS on success or else negative error code.

3.20.8 imrectangleTaskArray

```
IM_API IM_STATUS imrectangleTaskArray(im_job_handle_t job_handle,
                                      rga_buffer_t dst,
                                      im_rect *rect_array,
                                      int array_size,
                                      uint32_t color,
                                      int thickness);
```

> Add an Draw multiple border operation to the specified job through job_handle. The configuration parameters are the same as imrectangleArray.

【Note】 The width and height of the rect must be greater than or equal to 2

Parameter	Description
job_handle	**[required]** job handle
dst	**[required]** target image
rect_array	**[required]** image region array_ptr to fill specified color width and height of rect must be greater than or equal to 2
array_size	**[required]** size of region arrays.
color	**[required]** fill with color
thickness	**[required]** Thickness of lines that make up the rectangle. Negative values, like -1, mean that the function has to draw a filled rectangle.

Return IM_STATUS_SUCCESS on success or else negative error code.

3.20.9 immakeBorder

```
IM_API IM_STATUS immakeBorder(rga_buffer_t src,
                              rga_buffer_t dst,
                              int top,
                              int bottom,
                              int left,
                              int right,
                              int border_type,
                              int value = 0,
                              int sync = 1,
                              int acquir_fence_fd = -1,
                              int *release_fence_fd = NULL);
```

> According to the configured top/bottom/left/right pixels, draw a border to the input image and output it to the output target image buffer.

【Note】 The width and height of the rect must be greater than or equal to 2

Parameter	Description
src	**[required]** input source image
dst	**[required]** output target image
top	**[required]** number of top pixels
bottom	**[required]** number of bottom pixels
left	**[required]** number of left pixels
right	**[required]** number of right pixels
border_type	**[required]** Border type IM_BORDER_CONSTANT // iiiiii abcdefgh iiiiiii with some specified value 'i' IM_BORDER_REFLECT //fedcba abcdefgh hgfedcb IM_BORDER_WRAP //cdefgh abcdefgh abcdefg
value	**[optional]** the pixel value at which the border is filled
sync	**[optional]** wait until operation complete
acquire_fence_fd	**[required]** used in async mode, run the job after waiting foracquire_fence signal
release_fence_fd	**[required]** used in async mode, as a parameter of imsync()

Return IM_STATUS_SUCCESS on success or else negative error code.

3.21 Image Mosaic

3.21.1 immosaic

```
IM_API IM_STATUS immosaic(const rga_buffer_t image,
                          im_rect rect,
                          int mosaic_mode,
                          int sync = 1,
                          int *release_fence_fd = NULL);
```

> Mosaic masking the specified area of the image.

Parameter	Description
image	**[required]** souce image
rect	**[required]** image region to mosaic
mosaic_mode	**[required]** set mosaic mode IM_MOSAIC_8 IM_MOSAIC_16 IM_MOSAIC_32 IM_MOSAIC_64 IM_MOSAIC_128
sync	**[optional]** wait until operation complete
release_fence_fd	**[optional]**Used in async mode, as a parameter of imsync()

Return IM_STATUS_SUCCESS on success or else negative error code.

3.21.2 immosaicArray

```
IM_API IM_STATUS immosaicArray(const rga_buffer_t image,
                               im_rect *rect_array,
                               int array_size,
                               int mosaic_mode,
                               int sync = 1,
                               int *release_fence_fd = NULL);
```

> Mosaic masking the specified multiple area of the image.

Parameter	Description
image	**[required]** target image
rect_array	**[required]** image region array_ptr to mosaic
array_size	**[required]** size of region arrays.
mosaic_mode	**[required]** set mosaic mode IM_MOSAIC_8 IM_MOSAIC_16 IM_MOSAIC_32 IM_MOSAIC_64 IM_MOSAIC_128
sync	**[optional]** wait until operation complete
release_fence_fd	**[optional]**Used in async mode, as a parameter of imsync()

Return IM_STATUS_SUCCESS on success or else negative error code.

3.21.3 immosaicTask

```
IM_API IM_STATUS immosaicTask(im_job_handle_t job_handle,
                              const rga_buffer_t image,
                              im_rect rect,
                              int mosaic_mode);
```

> Add an image mosaic masking operation to the specified job through job_handle. The configuration parameters are the same as immosaic.

Parameter	Description
job_handle	**[required]** job handle
image	**[required]** target image
rect	**[required]** image region to mosaic
mosaic_mode	**[required]** set mosaic mode IM_MOSAIC_8 IM_MOSAIC_16 IM_MOSAIC_32 IM_MOSAIC_64 IM_MOSAIC_128

Return IM_STATUS_SUCCESS on success or else negative error code.

3.21.4 immosaicTaskArray

```
IM_API IM_STATUS immosaicTaskArray(im_job_handle_t job_handle,
                                   const rga_buffer_t image,
                                   im_rect *rect_array,
                                   int array_size,
                                   int mosaic_mode);
```

> Add multiple image mosaic masking operation to the specified job through job_handle. The configuration parameters are the same as immosaicArray.

Parameter	Description
job_handle	**[required]** job handle
image	**[required]** target image
rect_array	**[required]** image region array_ptr to mosaic
array_size	**[required]** size of region arrays.
mosaic_mode	**[required]** set mosaic mode IM_MOSAIC_8 IM_MOSAIC_16 IM_MOSAIC_32 IM_MOSAIC_64 IM_MOSAIC_128

Return IM_STATUS_SUCCESS on success or else negative error code.

3.22 Image Process

3.22.1 improcess

```
IM_STATUS improcess(rga_buffer_t src,
                    rga_buffer_t dst,
                    rga_buffer_t pat,
                    im_rect srect,
                    im_rect drect,
                    im_rect prect,
                    int acquire_fence_fd,
                    int *release_fence_fd,
                    im_opt_t *opt,
                    int usage);
```

> RGA image compound operation. Other APIs are developed based on this API, improcess can achieve more complex compound operations.
>
> Image processes are configured by usage.

usage definitions:

```
typedef enum {
    /* Rotation */
    IM_HAL_TRANSFORM_ROT_90     = 1 << 0,
    IM_HAL_TRANSFORM_ROT_180    = 1 << 1,
    IM_HAL_TRANSFORM_ROT_270    = 1 << 2,
    IM_HAL_TRANSFORM_FLIP_H     = 1 << 3,
    IM_HAL_TRANSFORM_FLIP_V     = 1 << 4,
    IM_HAL_TRANSFORM_FLIP_H_V   = 1 << 5,
```

```
    IM_HAL_TRANSFORM_MASK       = 0x3f,

    /*
     * Blend
     * Additional blend usage, can be used with both source and target configs.
     * If none of the below is set, the default "SRC over DST" is applied.
     */
    IM_ALPHA_BLEND_SRC_OVER     = 1 << 6,     /* Default, Porter-Duff "SRC over DST" */
    IM_ALPHA_BLEND_SRC          = 1 << 7,     /* Porter-Duff "SRC" */
    IM_ALPHA_BLEND_DST          = 1 << 8,     /* Porter-Duff "DST" */
    IM_ALPHA_BLEND_SRC_IN       = 1 << 9,     /* Porter-Duff "SRC in DST" */
    IM_ALPHA_BLEND_DST_IN       = 1 << 10,    /* Porter-Duff "DST in SRC" */
    IM_ALPHA_BLEND_SRC_OUT      = 1 << 11,    /* Porter-Duff "SRC out DST" */
    IM_ALPHA_BLEND_DST_OUT      = 1 << 12,    /* Porter-Duff "DST out SRC" */
    IM_ALPHA_BLEND_DST_OVER     = 1 << 13,    /* Porter-Duff "DST over SRC" */
    IM_ALPHA_BLEND_SRC_ATOP     = 1 << 14,    /* Porter-Duff "SRC ATOP" */
    IM_ALPHA_BLEND_DST_ATOP     = 1 << 15,    /* Porter-Duff "DST ATOP" */
    IM_ALPHA_BLEND_XOR          = 1 << 16,    /* Xor */
    IM_ALPHA_BLEND_MASK         = 0x1ffc0,

    IM_ALPHA_COLORKEY_NORMAL    = 1 << 17,
    IM_ALPHA_COLORKEY_INVERTED  = 1 << 18,
    IM_ALPHA_COLORKEY_MASK      = 0x60000,

    IM_SYNC                     = 1 << 19,
    IM_ASYNC                    = 1 << 26,
    IM_CROP                     = 1 << 20,    /* Unused */
    IM_COLOR_FILL               = 1 << 21,
    IM_COLOR_PALETTE            = 1 << 22,
    IM_NN_QUANTIZE              = 1 << 23,
    IM_ROP                      = 1 << 24,
    IM_ALPHA_BLEND_PRE_MUL      = 1 << 25,
} IM_USAGE;
```

Parameter	Description
src	**[required]** input imageA
dst	**[required]** output image
pat	**[required]** input imageB
srect	**[required]** src crop region
drect	**[required]** dst crop region
prect	**[required]** pat crop region
acquire_fence_fd	**[required]** Used in async mode, run the job after waiting foracquire_fence signal
release_fence_fd	**[required]** Used in async mode, as a parameter of imsync()
opt	**[required]** operation options typedef struct im_opt { int color; im_colorkey_range colorkey_range; im_nn_t nn; int rop_code; int priority; int core; } im_opt_t;
usage	**[required]** image operation usage

Return IM_STATUS_SUCCESS on success or else negative error code.

3.22.2 improcessTask

```
IM_API IM_STATUS improcessTask(im_job_handle_t job_handle,
                               rga_buffer_t src,
                               rga_buffer_t dst,
                               rga_buffer_t pat,
                               im_rect srect,
                               im_rect drect,
                               im_rect prect,
                               im_opt_t *opt_ptr,
                               int usage);
```

> Add an image compound operation to the specified job through job_handle. The configuration parameters are the same as improcess.

Parameter	Description
job_handle	**[required]** job handle
src	**[required]** input imageA
dst	**[required]** output image
pat	**[required]** input imageB
srect	**[required]** src crop region
drect	**[required]** dst crop region
prect	**[required]** pat crop region
acquire_fence_fd	**[required]** Used in async mode, run the job after waiting foracquire_fence signal
release_fence_fd	**[required]** Used in async mode, as a parameter of imsync()
opt	**[required]** operation options typedef struct im_opt { int color; im_colorkey_range colorkey_range; im_nn_t nn; int rop_code; int priority; int core; } im_opt_t;
usage	**[required]** image operation usage

Return IM_STATUS_SUCCESS on success or else negative error code.

3.23 Parameter Check

3.23.1 imcheck

```
IM_API IM_STATUS imcheck(const rga_buffer_t src, const rga_buffer_t dst,
                         const im_rect src_rect, const im_rect dst_rect,
                         const int mode_usage);
IM_API IM_STATUS imcheck_composite(const rga_buffer_t src, const rga_buffer_t dst, const rga_buffer_t pat,
```

```
                                    const im_rect src_rect, const im_rect dst_rect, const im_rect pat_rect,
                                    const int mode_usage);
```

> After RGA parameters are configured, users can use this API to verify whether the current parameters are valid and determine whether the hardware supports them based on the current hardware conditions.
>
> Users are advised to use this API only during development and debugging to avoid performance loss caused by multiple verification.

Parameter	Description
src	**[required]** input imageA
dst	**[required]** output image
pat	**[optional]** input imageB
srect	**[required]** src crop region
drect	**[required]** dst crop region
prect	**[optional]** pat crop region
usage	**[optional]** image operation usage

Return IM_STATUS_NOERROR on success or else negative error code.

3.24 Synchronous operation

3.24.1 imsync

```
IM_STATUS imsync(int fence_fd);
```

> RGA asynchronous mode requires this API to be called, passing the returned release_fence_fd as parameter.
>
> Other API enable asynchronous call mode when sync is set to 0, which is equivalent to glFlush in opengl. Further calls to imsync is equivalent to glFinish.

Parameter	Description
fence_fd	**[required]** fence_fd to wait

Return IM_STATUS_SUCCESS on success or else negative error code.

3.25 Thread Context Configuration

3.25.1 imconfig

```
IM_STATUS  imconfig(IM_CONFIG_NAME name, uint64_t value);
```

> The context for the current thread is configured through different configuration name, which will be the default configuration for the thread.
>
> The thread context configuration has a lower priority than the parameter configuration of the API. If no related parameters are configured for API, the local call uses the default context configuration. If related parameters are configured for API, the call uses the API parameter configuration.

Parameter	Description
name	**[required]** context config name： IM_CONFIG_SCHEDULER_CORE —— Specify the task processing core IM_CONFIG_PRIORITY —— Specify the task priority IM_CHECK_CONFIG —— Check enable
value	**[required]** config value IM_CONFIG_SCHEDULER_CORE : IM_SCHEDULER_RGA3_CORE0 IM_SCHEDULER_RGA3_CORE1 IM_SCHEDULER_RGA2_CORE0 IM_SCHEDULER_RGA3_DEFAULT IM_SCHEDULER_RGA2_DEFAULT IM_CONFIG_PRIORITY: 0 ~ 6 IM_CHECK_CONFIG: TRUE FALSE

> Note：Permissions of priority and core are very high. Improper operations may cause system crash or deadlock. Therefore, users are advised to configure them only during development and debugging. Users are not advised to perform this configuration in actual product

Return IM_STATUS_SUCCESS on success or else negative error code

4. Data Structure

This section describes the data structures involved in API in detail.

4.1 Overview

Data structure	Description
rga_buffer_t	image buffer information
im_rect	the actual operating area of the image
im_opt_t	image manipulation options
im_job_handle_t	RGA job handle
rga_buffer_handle_t	RGA driver image buffer handle
im_handle_param_t	buffer parameters of image to be imported
im_context_t	default context for the current thread
im_nn_t	operation point preprocessing parameters
im_colorkey_range	Colorkey range

4.2 Detailed Descriptions

4.2.1 rga_buffer_t

- **descriptions**

Buffer information of image with single channel.

- **path**

im2d_api/im2d_type.h

- **definitions**

```
typedef struct {
    void* vir_addr;                 /* virtual address */
    void* phy_addr;                 /* physical address */
    int fd;                         /* shared fd */
    rga_buffer_handle_t handle;     /* buffer handle */

    int width;                      /* width */
    int height;                     /* height */
    int wstride;                    /* wstride */
    int hstride;                    /* hstride */
    int format;                     /* format */

    int color_space_mode;           /* color_space_mode */
    int global_alpha;               /* global_alpha */
    int rd_mode;
} rga_buffer_t;
```

Parameter	Description
vir_addr	Virtual address of image buffer.
phy_addr	Contiguous physical address of the image buffer.
fd	File descriptor of image buffer DMA.
handle	Import handle corresponding to the image buffer of the RGA driver.
width	The width of the actual operating area of image,in pixels.
height	The height of the actual operating area of image,in pixels.
wstride	The stride of the image width, in pixels.
hstride	The stride of the image height, in pixels.
format	Image format.
color_space_mode	Image color space mode.
global_alpha	Global Alpha configuration.
rd_mode	The mode in which the current channel reads data.

- **Note**

Simply selects either one of vir_addr、phy_addr、fd、handle as the description of image buffer, if multiple values are assigned, only one of them is selected as the image buffer description according to the default priority, which is as follows：handle > phy_addr > fd > vir_addr.

4.2.2 im_rect

- **descriptions**

Describes the actual operation area of image with single channel.

- **path**

im2d_api/im2d_type.h

- **definitions**

```
typedef struct {
    int x;        /* upper-left x */
    int y;        /* upper-left y */
    int width;    /* width */
    int height;   /* height */
} im_rect;
```

Parameters	Description
x	The starting abscissa of the actual operation area of the image, in pixels.
y	The starting ordinate of the actual operating area of an image, in pixels.
width	The width of the actual operating area of the image, in pixels.
height	The height of the actual operating area of the image, in pixels.

- **Note**

The actual operating area cannot exceed the image size, i.e（x + width）<= wstride， （y + height） <= hstride。

4.2.3 im_opt_t

- **description**

Describes operation options of current image.

- **path**

im2d_api/im2d_type.h

- **definitions**

```
typedef struct im_opt {
    int color;                          /* color, used by color fill */
    im_colorkey_range colorkey_range;   /* range value of color key */
    im_nn_t nn;
    int rop_code;

    int priority;
    int core;
} im_opt_t;
```

Parameter	Description
color	Image color-fill configuration.
colorkey_range	Colorkey range configuration.
nn	Operation point preprocessing (quantization) configuration.
rop_code	ROP operation code configuration.
priority	Current task priority configuration.
core	Specify the hardware core of current task.

- **Note**

Permissions of priority and core are very high. Improper operations may cause system crash or deadlock. Therefore, users are advised to configure them only during development and debugging.Users are not advised to perform this configuration in actual product.

4.2.4 im_job_handle_t

- 说明

RGA jobhandle, used to identify the currently configured RGA job.

- 路径

im2d_api/im2d_type.h

- 定义

```
typedef uint32_t im_job_handle_t;
```

- 注意事项

After the configuration fails, imcancelJob must be used to release the current task handle to avoid memory leaks.

4.2.5 rga_buffer_handle_t

- **description**

RGA driver image buffer handle.

- **path**

im2d_api/im2d_type.h

- **definitions**

```
typedef int rga_buffer_handle_t;
```

- **Note**

When the buffer is used up, releasebuffer_handle must be used to release the memory to avoid memory leaks.

4.2.6 im_handle_param_t

- **description**

Describe parameters of the image buffer to be imported.

- **path**

im2d_api/im2d_type.h

- **definitions**

```
typedef struct rga_memory_parm im_handle_param_t;

struct rga_memory_parm {
    uint32_t width_stride;
    uint32_t height_stride;
    uint32_t format;
};
```

Parameter	Description
width_stride	Describes the horizontal stride of the image buffer to be imported, in pixels.
height_stride	Describes the vertical stride of the image buffer to be imported, in pixels.
format	Describes the format of the buffer of the image to be imported.

- **Note**

If the actual size of buffer memory is smaller than the configured size, the importbuffer_T API error occurs.

4.2.7 im_nn_t

- **description**

Parameter of operation point preprocessing (quantization).

- **path**

im2d_api/im2d_type.h

- **definitions**

```
typedef struct im_nn {
    int scale_r;                /* scaling factor on R channal */
    int scale_g;                /* scaling factor on G channal */
    int scale_b;                /* scaling factor on B channal */
    int offset_r;               /* offset on R channal */
    int offset_g;               /* offset on G channal */
    int offset_b;               /* offset on B channal */
} im_nn_t;
```

Parameter	Description
scale_r	Scaling factor on red channel.
scale_g	Scaling factor on green channel.
scale_b	Scaling factor on blue channel.
offset_r	Offset on red channel.
offset_g	Offset on green channel.
offset_b	Offset on blue channel.

- **Note**

null

4.2.8 im_colorkey_range

- **description**

Colorkey range.

- **path**

im2d_api/im2d_type.h

- **definitions**

```
typedef struct {
    int max;                    /* The Maximum value of the color key */
    int min;                    /* The minimum value of the color key */
} im_colorkey_range;
```

Parameter	Description
max	The Maximum value of the color key.
min	The minimum value of the color key.

- **Note**

null

5. Test Cases and Debugging Methods

> In order to make developers get started with the above new API more quickly, here by running demo and parsing the source code to help developers to understand and use the API.

5.1 Test File Description

> Input and output binary file for testing should be prepared in advance. The default source image file in RGBA8888 format is stored in directory /sample/sample_file.
>
> In Android system, the source image should be stored in /data/ directory of device，in Linux system, the source image should be stored in/usr/data directory of device. The file naming rules are as follows:

```
in%dw%d-h%d-%s.bin
out%dw%d-h%d-%s.bin

Example:
1280×720 RGBA8888 input image: in0w1280-h720-rgba8888.bin
1280×720 RGBA8888 output image: out0w1280-h720-rgba8888.bin
```

Parameter descriptions：

> The input is in ,the output is out.
> --->The first%d is the index of files, usually 0, used to distinguish files of the same format, width and height.
> --->The second%d is width, usually indicates virtual width.
> --->The third%d is height, usually indicates virtual height.
> --->The fourth%s is the name of format.
>
> Some common image formats for preset tests are as follows. You can view names of other formats in rgaUtils.cpp:

format（Android）	**format（Linux）**	**name**
HAL_PIXEL_FORMAT_RGB_565	RK_FORMAT_RGB_565	"rgb565"
HAL_PIXEL_FORMAT_RGB_888	RK_FORMAT_RGB_888	"rgb888"
HAL_PIXEL_FORMAT_RGBA_8888	RK_FORMAT_RGBA_8888	"rgba8888"
HAL_PIXEL_FORMAT_RGBX_8888	RK_FORMAT_RGBX_8888	"rgbx8888"
HAL_PIXEL_FORMAT_BGRA_8888	RK_FORMAT_BGRA_8888	"bgra8888"
HAL_PIXEL_FORMAT_YCrCb_420_SP	RK_FORMAT_YCrCb_420_SP	"crcb420sp"
HAL_PIXEL_FORMAT_YCrCb_NV12	RK_FORMAT_YCbCr_420_SP	"nv12"
HAL_PIXEL_FORMAT_YCrCb_NV12_VIDEO	/	"nv12"
HAL_PIXEL_FORMAT_YCrCb_NV12_10	RK_FORMAT_YCbCr_420_SP_10B	"nv12_10"

> The default resolution of input image of demo is 1280x720, format is RGBA8888, in0w1280-h720-rgba8888.bin source image should be prepared in advance in the /data or /usr/data directory, in1w1280-h720-rgba8888.bin source image should be additionally prepared in advance in the /data or /usr/data directory in image blending mode.

5.2 Debugging Method Description

> After running demo, print log as follows (in image copying, for example):
>
> Log is printed in Android system as follows:

```
# rgaImDemo --copy

librga:RGA_GET_VERSION:3.02,3.020000                    //RGA version
ctx=0x7ba35c1520,ctx->rgaFd=3                           //RGA context
Start selecting mode
im2d copy ..                                            //RGA running mode
GraphicBuffer check ok
GraphicBuffer check ok
lock buffer ok
open file ok                                            //src file status, if there is no corresponding file
in /data/ directory, an error will be reported here
unlock buffer ok
lock buffer ok
unlock buffer ok
copying .... successfully                               //indicates successful running
open /data/out0w1280-h720-rgba8888.bin and write ok      //output filename and directory
```

> Log is printed in Linux system as follows:

```
# rgaImDemo --copy

librga:RGA_GET_VERSION:3.02,3.020000                    //RGA version
ctx=0x2b070,ctx->rgaFd=3                                //RGA context
Rga built version:version:1.00
Start selecting mode
im2d copy ..                                            //RGA running mode
open file                                               //src file status, if there is no corresponding file
in /usr/data/ directory, an error will be reported here
copying .... Run successfully                           //indicates successful running
open /usr/data/out0w1280-h720-rgba8888.bin and write ok //output filename and directory
```

> To view more detailed logs of RGA running, the Android system can enable RGA configuration log print by setting vendor.rga.log (Android 8 and below is sys.rga.log):

```
setprop vendor.rga.log 1        enable RGA log print
logcat -s librga                enable and filter log print
setprop vendor.rga.log 0         cancel RGA log print
```

> In Linux system, you should open core/NormalRgaContext.h, set __DEBUG to 1 and recompile.

```
#ifdef LINUX

-#define __DEBUG 0
+#define __DEBUG 1
```

> Generally, the printed log is as follows, which can be uploaded to RedMine for analysis by relevant engineers of RK:
>
> Log is printed in Android system as follows:

```
 D librga  : <<<<-------- print rgaLog -------->>>>
 D librga  : src->hnd = 0x0 , dst->hnd = 0x0
 D librga  : srcFd = 11 , phyAddr = 0x0 , virAddr = 0x0
 D librga  : dstFd = 15 , phyAddr = 0x0 , virAddr = 0x0
 D librga  : srcBuf = 0x0 , dstBuf = 0x0
 D librga  : blend = 0 , perpixelAlpha = 1
 D librga  : scaleMode = 0 , stretch = 0;
 D librga  : rgaVersion = 3.020000  , ditherEn =0
 D librga  : srcMmuFlag = 1 , dstMmuFlag = 1 , rotateMode = 0
```

```
D librga  : <<<<-------- rgaReg -------->>>>
D librga  : render_mode=0 rotate_mode=0
D librga  : src:[b,0,e1000],x-y[0,0],w-h[1280,720],vw-vh[1280,720],f=0
D librga  : dst:[f,0,e1000],x-y[0,0],w-h[1280,720],vw-vh[1280,720],f=0
D librga  : pat:[0,0,0],x-y[0,0],w-h[0,0],vw-vh[0,0],f=0
D librga  : ROP:[0,0,0],LUT[0]
D librga  : color:[0,0,0,0,0]
D librga  : MMU:[1,0,80000521]
D librga  : mode[0,0,0,0]
```

> Log is printed in Linux system as follows:

```
render_mode=0 rotate_mode=0
src:[0,a681a008,a68fb008],x-y[0,0],w-h[1280,720],vw-vh[1280,720],f=0
dst:[0,a6495008,a6576008],x-y[0,0],w-h[1280,720],vw-vh[1280,720],f=0
pat:[0,0,0],x-y[0,0],w-h[0,0],vw-vh[0,0],f=0
ROP:[0,0,0],LUT[0]
color:[0,0,0,0,0]
MMU:[1,0,80000521]
mode[0,0,0,0,0]
gr_color_x [0, 0, 0]
gr_color_x [0, 0, 0]
```

5.3 Test Case Descriptions

- The test path is sample/im2d_API_demo. Developers can modify the demo configuration as required. It is recommended to use the default configuration when running demo for the first time.
- The compilation of test cases varies on different platforms. On the Android platform, the 'mm' command can be used to compile the test cases. On the Linux platform, when librga.so is compiled using cmake, the corresponding test cases will be generated in the same directory
- Import the executable file generated by compiling the corresponding test case into the device through adb, add the execution permission, execute demo, and check the printed log.
- Check the output file to see if it matches your expectations.

5.3.1 Apply for Image Buffer

> The demo provides two types of buffer for RGA synthesis: Graphicbuffer and AHardwareBuffer. The two buffers are distinguished by the macro USE_AHARDWAREBUFFER.

```
Directory: librga/samples/im2d_api_demo/Android.mk
 (line +15)

ifeq (1,$(strip $(shell expr $(PLATFORM_SDK_VERSION) \> 25)))
/*if USE_AHARDWAREBUFFER is set to 1 then use AHardwareBuffer, if USE_AHARDWAREBUFFER is set to 0 then use
Graphicbuffer*/
LOCAL_CFLAGS += -DUSE_AHARDWAREBUFFER=1
endif
```

5.3.1.1 Graphicbuffer

> Graphicbuffer is initialized, filled/emptied, and filling rga_buffer_t structure through three functions.

```
    /*Passing in width, height, and image formats of src/dst and initialize Graphicbuffer*/
```

```
    src_buf = GraphicBuffer_Init(SRC_WIDTH, SRC_HEIGHT, SRC_FORMAT);
    dst_buf = GraphicBuffer_Init(DST_WIDTH, DST_HEIGHT, DST_FORMAT);

    /*Fill/empty Graphicbuffer by enumerating FILL_BUFF/EMPTY_BUFF*/
    GraphicBuffer_Fill(src_buf, FILL_BUFF, 0);
    if(MODE == MODE_BLEND)
        GraphicBuffer_Fill(dst_buf, FILL_BUFF, 1);
    else
        GraphicBuffer_Fill(dst_buf, EMPTY_BUFF, 1);

    /*Fill rga_buffer_t structure: src、dst*/
    src = wrapbuffer_GraphicBuffer(src_buf);
    dst = wrapbuffer_GraphicBuffer(dst_buf);
```

5.3.1.2 AHardwareBuffer

> AHardwareBuffer is initialized, filled/emptied, and filling rga_buffer_t structure through three functions.

```
    /*Passing in width, height, and image formats of src/dst and initialize AHardwareBuffer*/
    AHardwareBuffer_Init(SRC_WIDTH, SRC_HEIGHT, SRC_FORMAT, &src_buf);
    AHardwareBuffer_Init(DST_WIDTH, DST_HEIGHT, DST_FORMAT, &dst_buf);

    /*Fill/empty AHardwareBuffer by enumerating FILL_BUFF/EMPTY_BUFF*/
    AHardwareBuffer_Fill(&src_buf, FILL_BUFF, 0);
    if(MODE == MODE_BLEND)
        AHardwareBuffer_Fill(&dst_buf, FILL_BUFF, 1);
    else
        AHardwareBuffer_Fill(&dst_buf, EMPTY_BUFF, 1);

    /*Fill rga_buffer_t structure: src、dst*/
    src = wrapbuffer_AHardwareBuffer(src_buf);
    dst = wrapbuffer_AHardwareBuffer(dst_buf);
```

5.3.2 Viewing Help Information

> Run the following command to obtain the help information about the test case:

```
rgaImDemo -h
rgaImDemo --help
rgaImDemo
```

> You can use demo according to the help information. The following information is printed:

```
rk3399_Android10:/ # rgaImDemo
librga:RGA_GET_VERSION:3.02,3.020000
ctx=0x7864d7c520,ctx->rgaFd=3

============================================================================================
   usage: rgaImDemo [--help/-h] [--while/-w=(time)] [--querystring/--querystring=<options>]
                    [--copy] [--resize=<up/down>] [--crop] [--rotate=90/180/270]
                    [--flip=H/V] [--translate] [--blend] [--cvtcolor]
                    [--fill=blue/green/red]
          --help/-h     Call help
          --while/w     Set the loop mode. Users can set the number of cycles by themselves.
          --querystring You can print the version or support information corresponding to the current version
of RGA according to the options.
                        If there is no input options, all versions and support information of the current
version of RGA will be printed.
```

```
                      <options>:
                      vendor          Print vendor information.
                      version         Print RGA version, and librga/im2d_api version.
                      maxinput        Print max input resolution.
                      maxoutput       Print max output resolution.
                      scalelimit      Print scale limit.
                      inputformat     Print supported input formats.
                      outputformat    Print supported output formats.
                      expected        Print expected performance.
                      all             Print all information.
        --copy        Copy the image by RGA.The default is 720p to 720p.
        --resize      resize the image by RGA.You can choose to up(720p->1080p) or down(720p->480p).
        --crop        Crop the image by RGA.By default, a picture of 300*300 size is cropped from (100,100).
        --rotate      Rotate the image by RGA.You can choose to rotate 90/180/270 degrees.

        --flip        Flip the image by RGA.You can choice of horizontal flip or vertical flip.
        --translate   Translate the image by RGA.Default translation (300,300).
        --blend       Blend the image by RGA.Default, Porter-Duff 'SRC over DST'.
        --cvtcolor    Modify the image format and color space by RGA.The default is RGBA8888 to NV12.
        --fill        Fill the image by RGA to blue, green, red, when you set the option to the corresponding
color.
==========================================================================================
```

> Parameter parsing is in the directory /librga/demo/im2d_api_demo/args.cpp.

5.3.3 Executing Demo in Loop

> Run the following command to loop demo. The loop command must precede all parameters. The number of cycles are of the type int and the default interval is 200ms.

```
rgaImDemo -w6 --copy
rgaImDemo --while=6 --copy
```

5.3.4 Obtain RGA Version and Support Information

> Run the following command to obtain the version and support information:

```
rgaImDemo --querystring
rgaImDemo --querystring=<options>
```

> If there is no input options, all versions and support information of current version of RGA will be printed.

```
options:
    =vendor               Print vendor information.
    =version              Print RGA version, and librga/im2d_api version.
    =maxinput             Print max input resolution.
    =maxoutput          Print max output resolution.
    =scalelimit         Print scale limit.
    =inputformat        Print supported input formats.
    =outputformat       Print supported output formats.
    =expected             Print expected performance.
    =all                    Print all information.
```

5.3.4.1 Code Parsing

> According to parameters of main() to print different information.

```
    /*Convert the parameters of main() into QUERYSTRING_INFO enumeration values*/
    IM_INFO = (QUERYSTRING_INFO)parm_data[MODE_QUERYSTRING];
    /*Print the string returned by querystring(), which is the required information*/
    printf("\n%s\n", querystring(IM_INFO));
```

5.3.5 Image Resizing

> Use the following command to test image resizing.

```
rgaImDemo --resize=up
rgaImDemo --resize=down
```

> This function must be filled withoptions as follows:

```
options:
    =up                 image resolution scale up to 1920x1080
    =down           image resolution scale down to 720x480
```

5.3.5.1 Code Parsing

> According to the parameters (up/down) of main() to choose to up(720p->1080p) or down(720p->480p), that is, for different scenarios, the buffer is re-initialized, emptied, or fills rga_buffer_t structure, and the rga_buffer_t structure that stores src and dst image data is passed to imresize().

```
    switch(parm_data[MODE_RESIZE])
    {
        /*scale up the image*/
        case IM_UP_SCALE :
            /*re-initialize Graphicbuffer to corresponding resolution 1920x1080*/
            dst_buf = GraphicBuffer_Init(1920, 1080, DST_FORMAT);
            /*empty the buffer*/
            GraphicBuffer_Fill(dst_buf, EMPTY_BUFF, 1);
            /*refill rga_buffer_t structure that stores dst data*/
            dst = wrapbuffer_GraphicBuffer(dst_buf);
            break;

        case IM_DOWN_SCALE :
            /*re-initialize Graphicbuffer to corresponding resolution 720x480**/
            dst_buf = GraphicBuffer_Init(720, 480, DST_FORMAT);
            /*empty the buffer*/
            GraphicBuffer_Fill(dst_buf, EMPTY_BUFF, 1);
            /*refill rga_buffer_t structure that stores dst data*/
            dst = wrapbuffer_GraphicBuffer(dst_buf);
            break;
    }
    /*pass src and dst of rga_buffer_t structure to imresize()*/
    STATUS = imresize(src, dst);
    /*print running status according to IM_STATUS enumeration values*/
    printf("resizing .... %s\n", imStrError(STATUS));
```

5.3.6 Image Cropping

> Test image cropping using the following command.

```
rgaImDemo --crop
```

> Options are not available for this feature. By default, crop the image within the coordinate LT(100,100)，RT(400,100)，LB(100,400)，RB(400,400).

5.3.6.1 Code Parsing

> Assign the size of clipped area in the im_rect structure that stores the src rectangle data, and pass the rga_buffer_t structure that stores the src and dst image data to imcrop().

```
    /*The coordinates of the clipped vertex are determined by x and y，the size of the clipped area is
determined by width and height*/
    src_rect.x      = 100;
    src_rect.y      = 100;
    src_rect.width  = 300;
    src_rect.height = 300;

    /*pass src and dst of src_rect structure and rga_buffer_t structure format to imcrop()*/
    STATUS = imcrop(src, dst, src_rect);
    /*print the running status according to the returned IM_STATUS enumeration values*/
    printf("cropping .... %s\n", imStrError(STATUS));
```

5.3.7 Image Rotation

> Test image rotation using the following command.

```
rgaImDemo --rotate=90
rgaImDemo --rotate=180
rgaImDemo --rotate=270
```

> This function must be filled with options, which are as follows:

```
options:
    =90                 Image rotation by 90°, exchange the width and height of output image resolution.
    =180                Image rotation by 180°, output image resolution unchanged.
    =270                Image rotation by 270°, exchange the width and height of output image resolution.
```

5.3.7.1 Code Parsing

> According to the arguments (up/down) of main() to choose the rotation degrees(90/180/270). IM_USAGE enumeration transformed from arguments values, together with the rga_buffer_t structure that stores src and dst image data is passed to imrotate().

```
    /*convert the parameters of main() into IM_USAGE enumeration values*/
    ROTATE = (IM_USAGE)parm_data[MODE_ROTATE];

    /*pass both IM_USAGE enumeration value that identifies the rotation degrees and src and dst of
rga_buffer_t structure format to imrotate()*/
    STATUS = imrotate(src, dst, ROTATE);
    /*print the running status according to the returned IM_STATUS enumeration values*/
    printf("rotating .... %s\n", imStrError(STATUS));
```

5.3.8 Image Mirror Flip

> Use the following command to test mirror flipping

```
rgaImDemo --flip=H
rgaImDemo --flip=V
```

> This function must be filled with options, which are as follows:

```
options:
    =H                  Image horizontal mirror flip.
    =V                  Image vertical mirror flip.
```

5.3.8.1 Code Parsing

> According to the arguments (H/V) of main() to choose the flipped direction, transform the arguments to IM_USAGE enumeration values, and the rga_buffer_t structure that stores src and dst image data is passed to imflip().

```
    /*convert the parameters of main() into IM_USAGE enumeration values*/
    FLIP = (IM_USAGE)parm_data[MODE_FLIP];

    /*pass both IM_USAGE enumeration value that identifies the flipped direction and src and dst of
rga_buffer_t structure format to imflip()*/
    STATUS = imflip(src, dst, FLIP);
    /*print the running status according to the returned IM_STATUS enumeration value*/
    printf("flipping .... %s\n", imStrError(STATUS));
```

5.3.9 Image Color Fill

> Use the following command to test the color fill.

```
rgaImDemo --fill=blue
rgaImDemo --fill=green
rgaImDemo --fill=red
```

This function must be filled with options. By default, fill the color of image within the coordinate LT(100,100)，RT(400,100)，LB(100,400)，RB(400,400), options are as follows:

```
options:
    =blue               Fill the image color as blue.
    =green          Fill the image color as green.
    =red                Fill the image color as red.
```

5.3.9.1 Code Parsing

> The filled color is determined according to the (blue/green/red) parameters of main(), and the size to be filled is assigned to the im_rect structure that stores the dst rectangle data, and the passed parameter is converted to the hexadecimal number of the corresponding color, which is passed to imfill() along with rga_buffer_t that stores the dst image data.

```
    /*Convert parameter of main() to hexadecimal number of the corresponding color*/
    COLOR = parm_data[MODE_FILL];

    /*The coordinates of clipping vertex are determined by x and y, and size of color-filled area is
determined by width and height*/
    dst_rect.x      = 100;
    dst_rect.y      = 100;
    dst_rect.width  = 300;
    dst_rect.height = 300;

    /*Pass dst_rect of im_rect format and hexadecimal number of the corresponding color together with src and
dst of rga_buffer_t format to imfill().*/
    STATUS = imfill(dst, dst_rect, COLOR);
    /*print the running status according to the returned IM_STATUS enumeration value*/
    printf("filling .... %s\n", imStrError(STATUS));
```

5.3.10 Image Translation

> Use the following command to test image translation.

```
rgaImDemo --translate
```

> This feature has no options. By default, the vertex (upper-left coordinate) is shifted to (300,300), that is, shifted 300 pixels to the right and 300 pixels down.

5.3.10.1 Code Parsing

> Assign the offset of translation to the im_rect that stores the src rectangle data, and pass the rga_buffer_t structure that stores the src and dst image data to imtranslate().

```
    /*The coordinates of vertices of translated image are determined by x and y*/
    src_rect.x = 300;
    src_rect.y = 300;

    /*pass the src_rect of im_rect format along with src and dst of rga_buffer_t format into imtranslate()*/
    STATUS = imtranslate(src, dst, src_rect.x, src_rect.y);
    /*print the running status according to the returned IM_STATUS enumeration value*/
    printf("translating .... %s\n", imStrError(STATUS));
```

5.3.11 Image Copying

> Use the following command to test image copying.

```
rgaImDemo --copy
```

> This feature has no options. The default copy resolution is 1280x720 and the format is RGBA8888.

5.3.11.1 Code Parsing

> Passing rga_buffer_t that stores src and dst image data to imcopy().

```
    /*passing src and dst of rga_buffer_t format to imcopy()*/
    STATUS = imcopy(src, dst);
    /*print the running status according to the returned IM_STATUS enumeration value*/
    printf("copying .... %s\n", imStrError(STATUS));
```

5.3.12 Image Blending

> Use the following command to test image blending.

```
rgaImDemo --blend
```

> This feature has no options. By default, the blending mode is IM_ALPHA_BLEND_DST.

5.3.12.1 Code Parsing

> Passing rga_buffer_t that stores src and dst image data to imblend().

```
    /*passing src and dst of rga_buffer_t format to imblend()*/
    STATUS = imblend(src, dst);
    /*print the running status according to the returned IM_STATUS enumeration value*/
    printf("blending .... %s\n", imStrError(STATUS));
```

5.3.13 Image Format Conversion

> Use the following command to test image format conversion.

```
rgaImDemo --cvtcolor
```

> This feature has no options. By default, images with resolution of 1280x720 will be converted from RGBA8888 to NV12.

5.3.13.1 Code Parsing

Assign the format to be converted in the format variable member of rga_buffer_t, and pass the rga_buffer_t structure that stores src and dst image data to imcvtcolor().

```
    /*Assign the format in the format variable member*/
    src.format = HAL_PIXEL_FORMAT_RGBA_8888;
    dst.format = HAL_PIXEL_FORMAT_YCrCb_NV12;

    /*passing the format to be converted and src and dst of rga_buffer_t format to  imcvtcolor()*/
    STATUS = imcvtcolor(src, dst, src.format, dst.format);
    /*print the running status according to the returned IM_STATUS enumeration value*/
    printf("cvtcolor .... %s\n", imStrError(STATUS));
```